



Руководство администратора

Browser Policy Manager 0.9.3

Содержание

Системные требования и границы поставки

- Минимальные системные требования для BPM
- Оценка готовности текущего одноузлового исходного развертывания
- Запись НА и границы отложенного производства

Развёртывание BPM из исходного кода в Linux

- Подготовка хоста Linux к развертыванию из исходного кода
- Настройка получения исходного кода в Linux
- Настройка параметров среды выполнения исходного кода Linux
- Проверка развертывания исходного кода Linux
- Установка BPM из исходников в Ubuntu 26.04 LTS
- Установка BPM из исходников в Debian 13.5
- Установка BPM из исходников в Fedora Linux 44
- Установка BPM из исходников в Linux Mint 22.3
- Установка BPM из исходников в стабильной ветке Manjaro

Развёртывание BPM из исходного кода через Windows WSL

- Подготовка Windows 10/11 к развертыванию исходного кода в WSL
- Настройка получения исходного кода BPM в WSL
- Настройка среды выполнения WSL и доступа к браузеру
- Проверка развертывания источника BPM через WSL

Эксплуатация и обновление развёртывания из исходного кода

- Проверка источников конфигурации DevOps
- Планирование хранения, журналов и резервных копий данных
- Проверка сети, CORS и ограничений безопасности
- Запись границ исходной операции
- Подготовка подтверждающих материалов перед обновлением исходного кода
- Обновление исходной версии и зависимостей
- Выполнение миграций и проверок обновления исходного кода
- Проверка обновления и применение условий остановки отката
- Проверка работоспособности и готовности BPM

Интеграция через API BPM

- Аудитория интеграции и границы API
- Поддерживаемые шаблоны интеграции
- Соглашения API

- Текущие ограничения API и ограничения безопасности
- Повторное использование примеров API и шаблонов среды DevOps
- Синхронизация данных жизненного цикла профиля
- Архивирование, восстановление, безвозвратное удаление и сброс профилей
- Импорт Firefox policies.json через API
- Экспорт канонического Firefox policies.json через API
- Проверка кандидата Firefox policies.json через API

Сценарии жизненного цикла и восстановление после сбоев

- Запуск сценария извлечения, сравнения и обновления контрольного продукта
- Запуск сценария импорта, проверки, экспорта и передачи соответствия требованиям
- Получение и просмотр профилей BPM из внешнего продукта управления
- Проверка и применение обновления профиля с параметром expected_revision
- Запуск импорта, проверки, экспорта и передачи метаданных соответствия
- Запуск продукта управления воротами с работоспособностью и готовностью BPM
- Восстановление после сбоев интеграции и запись доказательств аудита

Устранение неполадок и оценка готовности к эксплуатации

- Устранение неполадок при сбое запуска и недоступных зондах
- Устранение ошибок кэша схемы и ошибок проверки API
- Устранение неполадок импорта и экспорта Firefox
- Устранение проблем с базой данных и хранилищем
- Устранение неполадок сети WSL и устаревших зависимостей
- Устранение недоступности портала документации
- Проверка сетевой доступности и готовности к работе через прокси
- Планирование входных данных для мониторинга, резервного копирования и окон обновления

Помощник BPM по документации

- Эксплуатация помощника по документации в BPM 0.9.3
- Поддержка границы помощника по документации в BPM 0.9.3

Системные требования и границы поставки

Минимальные системные требования для BPM

Используйте эти минимальные требования для поддерживаемого запуска BPM из исходного кода в версии 0.9.3.

Базовая конфигурация BPM

Запускайте BPM из исходного кода в Linux x86_64: Ubuntu 26.04 LTS, Debian 13.5, Fedora Linux 44, Linux Mint 22.3 или актуальной стабильной ветви Manjaro. Windows 10 и Windows 11 поддерживаются только как хост для WSL 2 с этим Linux-сценарием. BPM не предоставляет нативную службу или установщик для Windows. Требуется CPython 3.14, два ядра CPU, 4 Гиб ОЗУ и 4 Гб свободного места для исходного кода, виртуального окружения, базы данных, журналов и сборки документации.

Браузер, сеть и доступ

Для веб-интерфейса BPM и портала документации используйте современный настольный браузер с включёнными JavaScript и HTTPS. Список поддерживаемых схем Firefox описывает совместимость данных политик, а не требование запускать BPM именно в Firefox. HTTPS и sudo нужны только при установке системных пакетов или зависимостей из исходного кода. Рабочий процесс должен выполняться от обычного пользователя Linux с правом записи в каталог BPM и настроенные каталоги данных; не запускайте приложение от root только ради его эксплуатации.

Помощник по документации в 0.9.3

Чат помощника по документации доступен как интерфейс для читателя, однако версия 0.9.3 не содержит локальной модели, RAG-индекса, эмбедингов, сгенерированных ответов, цитат или доступа к внешним источникам. Поэтому он не предъявляет дополнительных требований к CPU, ОЗУ, диску, GPU, NPU, облачной учётной записи или сети сверх базового сценария BPM.

На каждый отправленный вопрос чат показывает локализованное уведомление об обучении локальной модели. Для актуальной информации о продукте используйте независимый детерминированный поиск по документации.

Оценка готовности текущего одноузлового исходного развертывания

Решите, готов ли текущий экземпляр BPM с исходным кодом к управляемой работе с одним узлом, и назовите доказательства, подтверждающие это решение.

Развертывание исходного кода Linux или Windows 10/11 через WSL завершено, зависимости установлены, и предполагаемые значения BPM_DATABASE_URL, BPM_DOCUMENTATION_SITE_DIR и BPM_SCHEMA_CACHE_DIR известны.

BPM может быть подготовлен для работы с одним узлом в текущем состоянии. Эта оценка не является сертификатом повышения эффективности производства, проектом HA или приемкой комплексных услуг.

1. Запишите точную конфигурацию источника и среды выполнения.

Захватите `git rev-parse --short HEAD`, версию BPM, BPM_HOST, BPM_PORT, BPM_DATABASE_URL, BPM_DOCUMENTATION_SITE_DIR, BPM_SCHEMA_CACHE_DIR, а также информацию о том, запускается ли процесс с помощью `make dev` или команды `uvicorn app.main:app`, принадлежащей непосредственному владельцу.

2. Запускайте проверки работоспособности и готовности из той же границы сети, которую будут использовать пользователи.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Запишите результат как свидетельство GET /health и GET /health/ready. Проходящие проверки доказывают только жизнеспособность и готовность исходного процесса.

3. Прежде чем предоставить общий доступ к экземпляру, проверьте данные и документацию.

Подтвердите, что `data/bpm.db` — это предполагаемая база данных, что по крайней мере один профиль можно экспортировать как `policies.json` и что `/help/` открывает созданный артефакт документации или задокументированное недоступное состояние.

4. Классифицировать экземпляр как операцию источника текущего состояния.

Внимание:

Не помечайте экземпляр как упакованный, защищенный, сбалансированный по нагрузке или высокодоступный. Если внешние элементы управления добавляют эти свойства, запишите их как внешние обязанности и ограничьте свидетельства BPM работой с источником с одним узлом.

В примечании о готовности указано, что можно безопасно сделать сегодня из исходного кода, какие проверки пройдены и какие решения должны дожидаться распространения и поддержки во

время выполнения.

Прежде чем предоставлять доступ к экземпляру за пределами рабочей станции локального администратора или контролируемого хоста лаборатории, ознакомьтесь с разделом «Воздействие сети и готовность прокси-сервера».

Запись НА и границы отложенного производства

Укажите, какие возможности производства, НА, обратного прокси-сервера, восстановления и управления услугами отсутствуют сегодня и что можно подготовить для последующих проектных работ.

Примечания о готовности, доступности сети, резервном копировании/экспорте, мониторинге и окне обновления существуют для текущего экземпляра, запускаемого из исходного кода.

В этом разделе фиксируется граница между текущей работой с одним узлом источника и будущей производственной архитектурой. В нем намеренно открыто говорится о том, что запланировано, но еще не реализовано.

1. Назовите поддерживаемую текущую форму.

Напишите, что документация BPM Администратор/DevOps поддерживает развертывание исходного кода, ручную настройку, проверки работоспособности/готовности, примеры API, подтверждение резервного копирования вручную, проверки перестроения документации и окна обновления для одного узла.

2. Назовите отсутствующие возможности службы и прокси-сервера.

Перечислите пакетные службы, собственные сервисные модули Linux, службы Windows, официальные рецепты обратного прокси-сервера, TLS/рецепты прокси-сервера, управляемые секреты, конфигурацию шлюза аутентификации и усиление безопасности производства, которые отсутствуют в исходной документации BPM.

3. Отсутствует имя НА и возможности обновления.

Перечислите поддерживаемую кластеризацию НА, координацию нескольких узлов, руководство по привязке сеансов балансировщика нагрузки, блокировку общего хранилища, миграцию с нулевым временем простоя, последовательные обновления, автоматическое переключение при сбое и официальную автоматизацию восстановления как будущую работу, а не текущее поведение.

4. Преобразуйте будущие потребности в исходные данные для проектирования.

Внимание:

Не размещайте несколько исходных узлов за балансировщиком нагрузки и не вызывайте поддерживаемый результат НА. Сбор требуемых RTO/RPO, проверка резервной копии, владение TLS, владение секретами, ведение журнала, мониторинг и ожидания окна обновления в качестве входных данных для последующих задач распространения/выполнения.

Запись границы предотвращает неподдерживаемое производство, НА и заявки обратного прокси, сохраняя при этом полезную подготовительную работу для будущих выпусков.

Используйте эту запись при документации руководства QA и при утверждении будущих элементов установщика, службы, обратного прокси-сервера и НА невыполненных работ.

Развёртывание BPM из исходного кода в Linux

Подготовка хоста Linux к развёртыванию из исходного кода

Перед установкой BPM из извлечения из репозитория проверьте поддерживаемую границу развёртывания исходного кода, предварительные требования хоста и возможности отложенного производства.

Используйте хост Linux, на котором можно установить Python 3.14+, Git, инструменты сборки и curl. Этот модуль руководство предполагает получение исходного кода, а не пакетного установщика.

BPM можно запускать из исходного кода для разработки, проверки и контролируемой административной оценки. Упакованные установщики, собственные сервисные модули, официальные рецепты systemd, рецепты обратного прокси-сервера и TLS, кластеризация HA, управляемые секреты и усиление безопасности производства не включены в этот выпуск и не должны быть представлены как поддерживаемое поведение развёртывания.

1. Определите цель развёртывания и запишите, что это развёртывание Linux на основе исходного кода.

Ожидаемый результат: в примечаниях к эксплуатации указано, что BPM запускается из проверки Git и что установщики для конкретного дистрибутива отложены.

2. Установите или проверьте необходимые компоненты хоста.

Требуемые инструменты: Git, Python 3.14+, pip, make, curl и оболочка, которая может запускать `source .venv/bin/activate`. Команды репозитория используют локальную виртуальную среду и не требуют глобального пакета BPM.

3. Создайте рабочий каталог, отдельный от производственных данных и секретов.

Держите извлечение репозитория, `.venv` и локальные данные SQLite по контролируемому пути. Не храните настоящие секреты в примерах документации или зафиксированных файлах.

4. Прежде чем продолжить, подтвердите, что намеренно не описано.

Внимание:

Этот модуль руководство не предоставляет пакетный установщик, встроенную службу, поддерживаемый модуль systemd, рецепт обратного прокси-сервера, завершение TLS, топологию HA, управляемое секретное хранилище, автоматизацию резервного копирования или контрольный список для повышения безопасности производства.

Хост и операционные предположения готовы к поддерживаемому рабочему процессу проверки исходного кода BPM.

Продолжите тему проверки исходного кода Linux перед настройкой переменных времени выполнения.

Настройка получения исходного кода в Linux

Клонируйте репозиторий BPM, создайте виртуальную среду Python, установите зависимости разработки и подготовьте схему локальной базы данных.

Хост Linux подготовлен, и администратор имеет доступ к URL-адресу репозитория BPM и целевой ветке или тегу для версии 0.9.1.

Репозиторий README определяет поддерживаемую последовательность установки источника. Используйте локальные команды репозитория, чтобы при последующем устранении неполадок можно было сравнить среду с зафиксированными сценариями.

1. Клонируйте репозиторий и войдите в кассу.

```
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

Замените заполнители URL-адресом утвержденного внутреннего репозитория и ссылкой на выпуск.

2. Создайте и активируйте локальную среду Python.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

Виртуальная среда остается внутри кассы и не фиксируется.

3. Установите исходные зависимости BPM из извлечения.

```
pip install -e ".[dev]"
```

Редактируемая установка обеспечивает соответствие кода времени выполнения и тестов текущему дереву исходного кода.

4. Применить миграцию базы данных для выбранной рабочей базы данных.

```
alembic upgrade head
```

Внимание:

Выполняйте миграцию только для предполагаемой базы данных BPM. Для исходного рабочего процесса по умолчанию это локальная база данных SQLite, а не база данных производственной службы.

На касе активирована среда Python, установлены зависимости и перенесенная схема базы данных.

Настройте среду выполнения перед запуском процесса приложения.

Настройка параметров среды выполнения исходного кода Linux

Задайте URL-адрес базы данных, хост, порт, режим перезагрузки, CORS и путь к сайту документации, используемый процессом BPM, запускаемым из исходного кода.

Зависимости извлечения установлены, и для базы данных, которую вы планируете использовать, применена миграция.

BPM считывает переменные среды с префиксом BPM_, а также поддерживает локальный файл .env репозитория. URL-адрес базы данных по умолчанию — `sqlite+aiosqlite:///./data/bpm.db`, в котором данные хранятся в `data/bpm.db`.

1. Создайте локальный каталог данных и определите URL-адрес базы данных.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
```

Для повторяющихся запусков исходного кода одно и то же значение может храниться в локальном файле .env, который не зафиксирован.

2. Установите значения сети и перезагрузите локальный исходный процесс.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

Используйте петлю для локальной проверки. Привязка к общему интерфейсу — это оперативное решение за пределами границы исходного кода и руководство.

3. Просмотрите CORS и настройки сайта документации.

Исходные значения по умолчанию включают BPM_ENABLE_CORS, BPM_CORS_ALLOW_ORIGINS и BPM_DOCUMENTATION_SITE_DIR. Сохраняйте разрешающий CORS только для контролируемой локальной оценки и указывайте BPM_DOCUMENTATION_SITE_DIR на установленный сайт статической документации, если он доступен.

4. Запустите BPM из сценария репозитория.

```
make dev
```

Makefile запускает `uvicorn app.main:app --reload --port 8000`. Держите терминал переднего плана открытым, чтобы были видны сообщения запуска и обратные трассировки.

BPM запускается из исходного кода с явными локальными настройками среды выполнения.

Проверка работоспособности, готовности продукта UI и первые административные дымовые проверки.

Проверка развертывания исходного кода Linux

Используйте проверки работоспособности, Интернет UI, журналы и команду узкого тестирования, чтобы убедиться, что экземпляр BPM, запускаемый из исходного кода, можно использовать.

Процесс BPM запускается из хранилища репозитория с заданными настройками BPM_DATABASE_URL, хоста, порта и сайта документации.

Текущее приложение показывает работоспособность в /health и готовность в /health/ready. Эти зонды подтверждают только ответ работающего процесса; они не доказывают усиление защиты производства, соблюдение требований Firefox, готовность к резервному копированию или поведение HA.

1. Проверьте работоспособность и готовность хоста Linux.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Ожидаемые ответы: {"status": "ok"} и {"status": "ready", "ready": true}.

2. Откройте продукт UI и убедитесь, что маршрут библиотеки отвечает.

Используйте `http://127.0.0.1:8000/profiles`. Маршрут должен загружать интерфейс BPM или показывать намеренное состояние приложения, а не обратную трассировку сервера.

3. Проверьте журналы приоритетных приложений.

Проверьте терминал, на котором работает `make dev` или `uvicorn app.main:app --reload --port 8000`. Записывайте ошибки запуска, ошибки миграции, конфликты портов и предупреждения сайта документации, не копируя секретные данные в билеты.

4. Зафиксируйте границы результата проверки.

Внимание:

Прохождение проверок работоспособности, маршрута интерфейса и журналов переднего процесса не подтверждает готовность к промышленной эксплуатации, поведение обратного прокси, управляемое резервное копирование или применение политик Firefox во время работы.

У администратора есть доказательства того, что развертывание исходного кода Linux запускается, отвечает на запросы работоспособности/готовности, обслуживает маршрут UI и не имеет непосредственных блокировщиков приоритетного журнала.

Если проверка не удалась, сохраните команду, имена переменных среды, ответы зонда и соответствующие строки журнала вместе для устранения неполадок. Не расширяйте этот модуль руководство до инструкций по отложенной установке или производственной топологии.

Установка BPM из исходников в Ubuntu 26.04 LTS

Полная последовательность установки BPM 0.9.1 с системным Python 3.14.

Нужны Ubuntu 26.04 LTS, права sudo, доступ HTTPS и согласованная ссылка Git BPM 0.9.1 вместо <approved-0.9.1-ref>.

Это односерверный запуск из исходников для контролируемой проверки, а не установка системной службы.

1. Установите зависимости, получите исходники, создайте окружение, выполните миграцию.

```
. /etc/os-release
test "$ID" = "ubuntu" && test "$VERSION_ID" = "26.04"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make python3.14 python3.14-dev
python3.14-venv
export BPM_PYTHON=/usr/bin/python3.14
"$BPM_PYTHON" --version
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Остановитесь при любой ошибке или если версия Python не 3.14.x.

2. Запустите BPM в основном терминале.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

3. Во втором терминале проверьте зонды и интерфейс.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
```

```
curl -fsS http://127.0.0.1:8000/health/ready  
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

4. Соблюдайте границу остановки.

При сбое нажмите Ctrl+C, сохраните вывод и журналы, не редактируйте data/bpm.db и не выполняйте alembic downgrade.

Миграция и сборка документации завершены, зонды отвечают, Библиотека профилей доступна, журналы можно проверить.

Последовательность команд успешно проверена в чистом контейнере пользовательского пространства Ubuntu 26.04. Эта проверка не подтверждает нативную загрузку, ядро, systemd, рабочий стол, оборудование, межсетевой экран узла или промышленное развёртывание.

Установка BPM из исходников в Debian 13.5

Полная установка с проверенной по контрольной сумме пользовательской сборкой CPython 3.14.6.

Нужны Debian 13.5, sudo, HTTPS, 4 ГБ свободного места и согласованная ссылка BPM 0.9.1.

Системный Python 3.13 не заменяется: Python 3.14.6 устанавливается в \$HOME/.local.

1. Соберите Python и подготовьте BPM.

```
. /etc/os-release
test "$ID" = "debian" && test "$VERSION_ID" = "13"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlib1g-dev
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

2. Запустите BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

3. Проверьте результат во втором терминале.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

4. Остановитесь безопасно при сбое.

Нажмите Ctrl+C, сохраните контрольную сумму и журналы; не выполняйте понижение схемы БД и не удаляйте пользовательский Python до диагностики.

Python 3.14.6, BPM, миграция, зонды и интерфейс подготовлены.

Последовательность команд успешно проверена в чистом контейнере пользовательского пространства Debian 13.5. CPython сообщил об отсутствии необязательных модулей `_dbm` и `_zstd`, но полный рабочий процесс BPM завершился успешно. Проверка не подтверждает нативную загрузку, ядро, `systemd`, рабочий стол, оборудование, межсетевой экран узла или промышленное развёртывание.

Установка BPM из исходников в Fedora Linux 44

Полная установка BPM 0.9.1 с системным Python 3.14.

Нужны Fedora 44, sudo, HTTPS и согласованная ссылка BPM 0.9.1.

Процедура предназначена для запуска из исходников на одном узле, а не для systemd или промышленной среды.

1. Установите и подготовьте BPM.

```
. /etc/os-release
test "$ID" = "fedora" && test "$VERSION_ID" = "44"
sudo dnf install -y ca-certificates curl gcc gcc-c++ git make python3 python3-devel
export BPM_PYTHON=/usr/bin/python3
"$BPM_PYTHON" --version
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

2. Запустите BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

3. Проверьте зонды и интерфейс.

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

4. Остановитесь безопасно.

При сбое нажмите Ctrl+C, сохраните вывод DNF, pip, миграции, сборки и журналов; не понижайте схему БД.

ВРМ мигрирован, документация собрана, зонды и интерфейс отвечают.

Последовательность команд успешно проверена в чистом контейнере пользовательского пространства Fedora 44. Эта проверка не подтверждает нативную загрузку, ядро, systemd, рабочий стол, оборудование, межсетевой экран узла или промышленное развёртывание.

Установка BPM из исходников в Linux Mint 22.3

Полная установка с проверенной пользовательской сборкой CPython 3.14.6.

Нужны Linux Mint 22.3, sudo, HTTPS, 4 ГБ места и согласованная ссылка BPM 0.9.1.

Системный Python не заменяется; CPython устанавливается в \$HOME/.local.

1. Соберите Python и подготовьте BPM.

```
. /etc/os-release
test "$ID" = "linuxmint" && test "$VERSION_ID" = "22.3"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlib1g-dev
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

2. Запустите BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

3. Проверьте результат.

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

4. Остановитесь безопасно.

При сбое нажмите Ctrl+C, сохраните контрольную сумму и журналы и не понижайте схему БД.

Python 3.14.6, BPM, документация, зонды и интерфейс подготовлены.

Последовательность команд успешно проверена в чистом контейнере пользовательского пространства Linux Mint 22.3. CPython сообщил об отсутствии необязательных модулей `_dbm` и `_zstd`, но полный рабочий процесс BPM завершился успешно. Проверка не подтверждает нативную загрузку, ядро, systemd, рабочий стол, оборудование, межсетевой экран узла или промышленное развёртывание.

Установка BPM из исходников в стабильной ветке Manjaro

Полная установка после обновления всей стабильной ветки и проверки Python 3.14.

Нужна стабильная ветка Manjaro не старше обновления 2026-06-26, sudo, HTTPS и согласованная ссылка BPM 0.9.1.

Manjaro обновляется непрерывно, поэтому в свидетельстве обязательны ветка и полностью обновлённое состояние пакетов.

1. Обновите систему и подготовьте BPM.

```
. /etc/os-release
test "$ID" = "manjaro"
test "$(pacman-mirrors -G)" = "stable"
sudo pacman -Syu --needed --noconfirm base-devel ca-certificates curl git python python-pip
pacman-mirrors -G
uname -r
export BPM_PYTHON=/usr/bin/python
"$BPM_PYTHON" --version
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Остановитесь, если ветка не stable, полное обновление не удалось или Python не 3.14.x.

2. Запустите BPM.

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

3. Проверьте результат.

```
cd browser-policy-manager
source .venv/bin/activate
```

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

4. Остановитесь безопасно.

При сбое нажмите Ctrl+C, сохраните сведения о ветке, пакетах и журналы; не выполняйте частичное понижение пакетов или схемы БД.

BPM установлен после полного обновления, документация собрана, зонды и интерфейс отвечают.

Последовательность команд успешно проверена в чистом контейнере пользовательского пространства стабильной ветки Manjaro с фиксацией ветки и полного состояния пакетов. Проверка не подтверждает нативную загрузку, ядро, systemd, рабочий стол, оборудование, межсетевой экран узла или промышленное развёртывание.

Развёртывание BPM из исходного кода через Windows WSL

Подготовка Windows 10/11 к развёртыванию исходного кода в WSL

Включите WSL, выберите поддерживаемый дистрибутив Linux и подтвердите, что BPM будет работать внутри Linux, а не как собственная служба Windows.

Используйте Windows 10 или Windows 11 с разрешением на включение WSL 2, установите дистрибутив Linux, например Ubuntu LTS, и запускайте команды как в PowerShell, так и в оболочке WSL.

В этом модуле руководство описано развёртывание исходного кода BPM через WSL. Поддерживаемый процесс приложения по-прежнему является рабочим процессом исходного кода Linux внутри WSL; В этом выпуске не представлены собственные установщики Windows, службы Windows, пакеты MSI, запланированные задачи, хостинг IIS и средства управления производственными службами Windows.

1. Подтвердите доступность WSL от PowerShell.

```
wsl --status  
wsl --list --verbose
```

Ожидаемый результат: WSL установлен, WSL 2 доступен, выбранный дистрибутив Linux виден.

2. Установите WSL и одобренный дистрибутив Linux, если они отсутствуют.

```
wsl --install  
wsl --install -d Ubuntu
```

Используйте имя распространения, одобренное вашей организацией. Перезагрузитесь, если Windows запросит это, затем создайте пользователя Linux при первом запуске дистрибутива.

3. Запишите границу дистрибутива и файловой системы.

Поместите BPM в файловую систему WSL Linux, например /home/<linux-user>/browser-policy-manager. Не запускайте проверку из /mnt/c, поскольку семантика файловой системы Windows и антивирусное сканирование могут сделать среду Python, просмотр файлов и разрешения ненадежными.

4. Подтвердите, что этот путь Windows еще не поддерживает.

Внимание:

Это не собственная установка Windows. руководство не предоставляет упаковку MSI/EXE, службу Windows, модель учетной записи службы, интеграцию средства просмотра событий, политику брандмауэра Windows, обратный прокси-сервер/рецепт TLS, топологию HA, управляемые секреты, автоматизацию резервного копирования или усиление безопасности производства.

Windows и WSL готовы к развертыванию исходного кода BPM на базе Linux с четкими границами платформы и файловой системы.

Продолжайте, открыв оболочку дистрибутива WSL и настроив получение исходного кода в домашнем каталоге Linux.

Настройка получения исходного кода BPM в WSL

Установите необходимые компоненты Linux в WSL, клонируйте BPM в файловую систему Linux, создайте виртуальную среду, установите зависимости и примените миграции.

Дистрибутив Linux WSL 2 установлен и открыт, и у администратора есть утвержденный URL-адрес репозитория BPM и ветка или тег 0.9.1.

Все команды в этом разделе выполняются внутри оболочки WSL Linux, если в шаге явно не указано PowerShell. Это обеспечивает согласованность настройки с руководством развертывания исходного кода Linux и позволяет избежать неоднозначности пути к собственному пути Windows.

1. Установите необходимые компоненты Linux из дистрибутива WSL.

```
sudo apt-get update
sudo apt-get install -y git make curl python3 python3-venv python3-pip
python3 --version
```

Используйте менеджер пакетов и имена пакетов, одобренные для выбранного вами дистрибутива WSL.

2. Клонировать BPM в домашнюю файловую систему Linux.

```
cd ~
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

Не размещайте рабочую проверку в /mnt/c для поддерживаемого исходного рабочего процесса WSL.

3. Создайте и активируйте локальную среду Python.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Если дистрибутив предоставляет Python только как python3, создайте среду с python3 -m venv .venv, а затем используйте активированную среду.

4. Примените миграцию базы данных внутри проверки WSL.

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Внимание:

Файл SQLite — это файл Linux внутри WSL. Не редактируйте и не перемещайте `data/bpm.db` из инструментов Windows во время работы BPM.

Для проверки WSL необходимы необходимые компоненты Linux, локальная виртуальная среда репозитория, установленные зависимости и перенесенная локальная база данных.

Перед запуском BPM настройте переменные времени выполнения и доступ к браузеру.

Настройка среды выполнения WSL и доступа к браузеру

Установите переменные времени выполнения BPM внутри WSL, выберите безопасный адрес привязки, запустите исходный процесс и определите URL-адрес браузера из Windows.

Извлечение BPM, виртуальная среда, зависимости и миграции готовы внутри файловой системы WSL Linux.

BPM считывает переменные среды BPM_ из процесса Linux. Современный WSL обычно пересылает localhost в Windows, но WSL IP можно проверить, если доступ к браузеру или поведение брандмауэра неясны.

1. Установите локальные переменные времени выполнения в оболочке WSL.

```
source .venv/bin/activate
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
export BPM_HOST="0.0.0.0"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

BPM_HOST="0.0.0.0" позволяет браузеру Windows получать доступ к процессу WSL посредством пересылки на локальный хост. Используйте 127.0.0.1 только в том случае, если браузер также находится внутри WSL или пересылка проверена.

2. Запустите BPM из проверки WSL.

```
make dev
```

Makefile запускает `uvicorn app.main:app --reload --port 8000`. Держите терминал WSL открытым для сообщений запуска, обратных трассировок и выходных данных средства наблюдения за файлами.

3. Определите URL-адрес браузера Windows.

```
hostname -I
```

Сначала попробуйте `http://localhost:8000/profiles` из Windows. Если переадресация локального хоста недоступна, используйте первый WSL IP, сообщенный `hostname -I`, например `http://<wsl-ip>:8000/profiles`.

4. Соблюдайте четко определенные границы безопасности.

Внимание:

Открытие порта WSL для других хостов, изменение правил брандмауэра Windows, добавление TLS или размещение обратного прокси-сервера перед BPM выходят за рамки этой темы развертывания исходного кода WSL и должны дожидаться рекомендаций по готовности к работе.

Исходный процесс BPM выполняется внутри WSL и имеет определенный путь доступа к браузеру Windows.

Проверьте работоспособность, готовность, доступ UI и режимы сбоя, характерные для WSL.

Проверка развертывания источника BPM через WSL

Проверяйте зонды из WSL и Windows, подтверждайте маршрут UI, просматривайте журналы и устраняйте распространенные проблемы с сетью и файловой системой WSL.

BPM запускается из проверки WSL с заданными настройками BPM_DATABASE_URL, хоста, порта и сайта документации.

Проверка должна подтвердить как ответ процесса Linux, так и путь доступа к браузеру Windows. Он не сертифицирует встроенную установку Windows, усиление безопасности производства, поведение обратного прокси-сервера, готовность к резервному копированию или принудительное выполнение Firefox во время выполнения.

1. Проверьте работоспособность и готовность внутри WSL.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Ожидаемые ответы: {"status": "ok"} и {"status": "ready", "ready": true}.

2. Проверьте те же зонды из Windows PowerShell.

```
Invoke-WebRequest http://localhost:8000/health
Invoke-WebRequest http://localhost:8000/health/ready
```

Если localhost не работает, повторите попытку с WSL IP из hostname -I. Запишите, какой адрес удался.

3. Откройте маршрут UI в браузере Windows.

Используйте `http://localhost:8000/profiles` или `http://<wsl-ip>:8000/profiles`. Маршрут библиотеки должен загружать интерфейс BPM или преднамеренное пустое состояние или состояние ошибки, а не обратную трассировку.

4. Устраните неполадки, связанные с WSL, перед изменением кода BPM.

В случае сбоя соединения убедитесь, что `make dev` все еще работает, проверьте BPM_HOST, запустите `wsl --list --verbose` и обновите WSL IP после перезапуска распространения. В случае медленного просмотра файлов или ошибок разрешения переместите извлечение из `/mnt/c` в `/home/<linux-user>`. Запускайте только после того, как путь к среде и сетевой маршрут станут стабильными.

Внимание:

Прохождение проверок работоспособности через WSL не подтверждает готовность к работе, поведение служб Windows, управляемое резервное копирование или поддержку НА.

У администратора есть доказательства того, что BPM запускается внутри WSL, отвечает на запросы, доступен из Windows и не имеет непосредственных блокировщиков сети или файловой системы WSL.

Храните выходные данные PowerShell, команды оболочки WSL, имена переменных BPM_*, ответы зондов и журналы приоритетного режима вместе для устранения неполадок без копирования секретов.

Эксплуатация и обновление развёртывания из исходного кода

Проверка источников конфигурации DevOps

Определите текущие источники конфигурации BPM, запишите эффективные значения времени выполнения и отделите обычные настройки от неподдерживаемых ожиданий управления секретами.

BPM развёртывается из исходного кода в Linux или через WSL, и администратор может проверять среду оболочки, дополнительный файл `.env` и извлекать исходный код.

BPM загружает настройки из переменных среды с префиксом `BPM_` и из корневого файла `.env` репозитория. Модель конфигурации находится в документированные значения по умолчанию. В этом выпуске не определены секретное хранилище приложений, хранилище учетных данных, настройка токена API, настройка ключа шифрования или профиль рабочей конфигурации.

1. Перечислите источники конфигурации, использованные для этого запуска.

Запишите, получены ли значения из экспорта оболочки, локального `.env` или значений по умолчанию в документированные значения по умолчанию. Сохраняйте `.env` локальным для кассы и не фиксируйте его.

2. Запишите настройки времени выполнения, влияющие на поведение процесса.

Включает `BPM_APP_NAME`, `BPM_APP_VERSION`, `BPM_DEBUG`, `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_API_PREFIX`, `BPM_DEFAULT_LOCALE`, `BPM_SUPPORTED_LOCALES`, `BPM_I18N_DIR` и `BPM_DOCUMENTATION_SITE_DIR`. Значения по умолчанию — это значения по умолчанию для локального источника, а не для производственной политики.

3. Записывайте данные, схему и настройки сети отдельно.

Отслеживать `BPM_DATABASE_URL`, `BPM_DB_ECHO`, `BPM_ENABLE_CORS`, `BPM_CORS_ALLOW_ORIGINS`, `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` и `BPM_SCHEMA_HTTP_TIMEOUT`. Рассматривайте базу данных URL-адреса и сетевое происхождение как оперативные доказательства, даже если они не содержат пароля.

4. Не храните секреты в примерах конфигурации BPM.

Внимание:

Документация по развёртыванию исходного кода BPM не предоставляет настройки аутентификации, управляемых секретов, конфигурации шифрования при хранении, ротации токенов, изоляции клиентов или гарантий редактирования секретов. Если вашей среде нужны эти элементы управления, запишите их как внешние элементы управления платформой, а не функции, поддерживаемые BPM.

В развёртывании имеется проверенная инвентарная конфигурация, которая соответствует текущим настройкам источника и не предполагает неподдерживаемого поведения по управлению секретами.

Используйте темы хранения, сети и операционных границ, чтобы проанализировать влияние записанных значений, прежде чем выставлять BPM за пределы контролируемой локальной среды.

Планирование хранения, журналов и резервных копий данных

Найдите базу данных развертывания источника, кэш схемы, сайт документации, журналы приоритетного режима и свидетельства резервного копирования вручную, не требуя поддержки управляемого резервного копирования или восстановления.

Инвентаризация конфигурации развертывания включает `BPM_DATABASE_URL`, `BPM_SCHEMA_CACHE_DIR` и `BPM_DOCUMENTATION_SITE_DIR`.

При развертывании источника по умолчанию данные приложения хранятся в SQLite по адресу `sqlite+aiosqlite:///./data/bpm.db`, что соответствует `data/bpm.db` в корне репозитория. Файлы кэша схемы по умолчанию находятся в `app/schemas/mozilla`, а установленный артефакт документации считывается из `app/documentation/site`, если он присутствует.

1. Определите файлы и каталоги, в которых сохраняется рабочее состояние.

Запишите путь к хранилищу: `data/`, `data/bpm.db`, `app/schemas/mozilla` и `app/documentation/site`. В WSL храните эти пути в файловой системе Linux, а не в `/mnt/c`.

2. Определите этап ручного резервного копирования доказательств перед рискованной работой.

Остановите основной процесс BPM, скопируйте `data/bpm.db` в контролируемое резервное хранилище и запишите временную метку, исходную версию, URL-адрес базы данных, размер файла и контрольную сумму. BPM не обеспечивает автоматическое резервное копирование, восстановление на определенный момент времени или протестированную оркестрацию восстановления.

3. Захват журналов текущего исходного процесса.

Исходный код запускает запись сообщений запуска, выходных данных доступа Uvicorn, обратных трассировок, ошибок миграции, конфликтов портов и предупреждений сайта документации на терминал переднего плана, стандартный вывод (`stdout`) или стандартный поток ошибок (`stderr`). В этом выпуске не описаны выделенный каталог журналов приложений BPM, политика ротации журналов, запрос ID, журнал аудита или интеграция средства просмотра событий.

4. Сопоставьте свидетельство резервного копирования файла с свидетельством экспорта, когда передача профиля имеет значение.

Используйте существующие примеры экспорта API для экспорта соответствующих документов `Firefox policies.json` и сохранения статуса ответа, профиля ID, версии, канала схемы и хеша экспортированного документа. Не рассматривайте свидетельство экспорта как полное восстановление базы данных.

Администратор знает, где находится текущее состояние развертывания исходного кода и какие доказательства можно собрать вручную перед обновлением или устранением неполадок.

Прежде чем изменять исходные версии или схемы базы данных, используйте будущий модуль руководство обновления из исходного кода и остановитесь, когда свидетельство резервного

копирования или восстановления окажется неполным.

Проверка сети, CORS и ограничений безопасности

Проверьте хост, порт, перезагрузку, проверки работоспособности, значения по умолчанию CORS и ограничения безопасности, прежде чем сделать экземпляр BPM, запускаемый исходным кодом, доступным.

Исходный процесс можно запустить с помощью `make dev`, а администратор может изменить `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_ENABLE_CORS` и `BPM_CORS_ALLOW_ORIGINS`.

Текущие значения по умолчанию оптимизированы для работы с локальным источником: `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `BPM_RELOAD="true"`, `BPM_ENABLE_CORS="true"` и разрешительные источники, эквивалентные `["*"]`. Эти значения по умолчанию не являются жесткой политикой воздействия на сеть.

1. Выберите самый узкий адрес привязки для текущей среды.

Используйте `127.0.0.1` для локальных проверок Linux. Используйте `0.0.0.0` только в том случае, если этого требует переадресация браузера WSL или контролируемая лабораторная сеть. Запишите причину любой привязки без обратной связи.

2. Оперативный доступ к воротам с помощью датчиков работоспособности и готовности.

Используйте `GET /health` и `GET /health/ready` перед примерами API или дымовыми проверками UI. Успешные проверки показывают только жизнеспособность/готовность процесса; они не подтверждают статус резервной копии базы данных, соблюдение требований Firefox, аутентификацию, авторизацию или производственную топологию.

3. Ограничить CORS для любых общих оценок.

Для локальных экспериментов разрешающее значение `BPM_CORS_ALLOW_ORIGINS` может быть приемлемым. Для общего хоста укажите явное происхождение и запишите, кому принадлежит происхождение браузера. В документации BPM не определены средства защиты CORS, CSRF, аутентификация API или изоляция клиента.

4. Не допускайте включения в утверждение BPM неподдерживаемых мер безопасности.

Внимание:

В этом выпуске не реализовано завершение TLS, рецепты обратного прокси-сервера, политика брандмауэра, ограничение скорости, подписание запросов, ведение журнала аудита, установка SSO, OAuth/OIDC, управление доступом на основе ролей, сканирование уязвимостей или отработка отказа HA. Задokumentируйте их как внешние элементы управления, если они добавлены в вашей среде.

Исходное состояние сети BPM документируется как контролируемое локальное или лабораторное воздействие, а не как производственная архитектура безопасности.

Если зонды не удалось или раскрытие неясно, воспользуйтесь темами проверки развертывания источника, прежде чем продолжить примеры интеграции API.

Запись границ исходной операции

Создайте записку о передаче операций, в которой указывается пользователь среды выполнения, модель процесса, поддерживаемые проверки и возможности отложенного производства.

Для экземпляра, запускаемого из исходного кода, завершены проверки конфигурации, хранилища, резервного копирования, сети и CORS.

Документация администратора BPM/DevOps в настоящее время поддерживает операцию запуска исходного кода для контролируемой оценки. Он еще не предоставляет установщики дистрибутива, собственные службы, официальную демонизацию, производственную топологию или полные рабочие книги задач для каждого элемента управления предприятия.

1. Запишите пользователя среды выполнения и модель процесса.

Определите пользователя оболочки, которому принадлежит проверка, `.venv`, `data/` и запущенный процесс `make dev` или `uvicorn app.main:app --reload --port 8000`. Не запрашивайте поддерживаемую учетную запись службы, модуль `systemd`, службу Windows или политику перезапуска, если ваша внешняя платформа не предоставляет и не владеет ими.

2. Запишите проверки, которые поддерживаются сегодня.

Включите исходную версию, активные значения конфигурации, статус миграции, дым маршрута `/health`, `/health/ready`, UI, пример состояния API, проверку основного журнала и доказательства резервного копирования вручную. Свяжите доказательства API с повторно используемыми примерами API, а не дублируйте процедуры конечных точек.

3. Запишите отложенные возможности как блокирующие или внешние обязанности.

Вызовите пакетные установщики, собственные службы Linux или Windows, обратный прокси-сервер и рецепты TLS, кластеризацию HA, управляемые секреты, конфигурацию аутентификации, автоматизацию резервного копирования, оркестрацию восстановления, панели мониторинга, ротацию журналов и усиление безопасности производства, что не предусмотрено исходной документацией BPM.

4. Прикрепите ссылки на восстановление, доступные пользователю, без передачи права собственности.

Если оператору необходимо понять симптомы UI, обратитесь к инструкциям по устранению неполадок, ориентированным на пользователя, например восстановлению соединения с продуктом. Сохраните развертывание, API, резервное копирование и владение сетью в Руководстве администратора/DevOps.

В записке о передаче указано, что может поддерживать текущее развертывание исходного кода, какие доказательства это подтверждают и какие производственные обязанности остаются за пределами BPM.

Используйте это граничное примечание в качестве отправной точки для выполнения руководства обновления из исходного кода и последующих задач документирования рабочей/HA/обратного прокси-сервера.

Подготовка подтверждающих материалов перед обновлением исходного кода

Соберите исходную версию, конфигурацию, резервную копию базы данных, данные экспорта профиля и условия остановки, прежде чем переносить BPM на более новую исходную версию.

BPM запускается из исходного кода Linux или WSL, и администратор выполнил проверку конфигурации, хранения, резервного копирования и операционных границ.

Процедура обновления из исходного кода изменяет код, зависимости, схему базы данных и ожидаемые выходные данные документации. BPM не обеспечивает автоматический откат, гарантии перехода на более раннюю версию базы данных, управляемое резервное копирование или пакетные инструменты обновления. Относитесь к сбору доказательств как к обязательному этапу подготовки к обновлению.

1. Остановите основной процесс BPM и запишите текущее исходное состояние.

```
git status --short
git rev-parse --short HEAD
```

Продолжайте только тогда, когда понятно состояние рабочего дерева и текущая исходная версия записана как ссылка на откат.

2. Запишите действующие пути конфигурации и среды выполнения.

Захватите активные `BPM_DATABASE_URL`, `BPM_DOCUMENTATION_SITE_DIR`, `BPM_SCHEMA_CACHE_DIR`, `BPM_HOST`, `BPM_PORT` и укажите, получены ли значения из экспорта оболочки или `.env`. Не копируйте секреты в примечание об обновлении.

3. Создайте свидетельство резервного копирования базы данных вручную, пока BPM остановлен.

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

Запишите временную метку, текущую версию, URL-адрес базы данных, размер файла и контрольную сумму. Если при развертывании не используется `data/bpm.db`, вместо этого создайте резервную копию пути, определенного из `BPM_DATABASE_URL`.

4. Экспортируйте данные критического профиля перед изменением кода.

Используйте существующие примеры экспорта API, чтобы сохранить необходимые документы `Firefox policies.json` и записать статус ответа, профиль ID, версию, канал схемы и хэш экспортированного документа.

Внимание:

Если резервная копия базы данных или свидетельства экспорта отсутствуют, остановите обновление. Не заменяйте отсутствующие доказательства резервной копии предположением, что откат Git или alembic downgrade будет безопасным.

Обновление содержит проверенный набор доказательств перед изменением и известную ссылку на предыдущий источник.

Приступайте к обновлению исходной версии и зависимостей только после того, как набор доказательств будет завершен и проверен.

Обновление исходной версии и зависимостей

Переместите извлечение в утвержденную новую версию и переустановите среду Python, не скрывая зависимости или сбои рабочего дерева.

Набор свидетельств перед обновлением существует, предыдущая версия записана, и известна утвержденная целевая ветвь, тег или фиксация.

Обновления исходного кода BPM — это операции в среде Git и Python, выполняемые в существующем извлечении. Эта процедура не устанавливает пакетный дистрибутив, встроенную службу или менеджер обновления рабочей среды.

1. Получите утвержденную ссылку на источник.

```
git fetch --tags
git status --short
```

Остановиться, если локальные изменения оказались неожиданными. Сохраняйте изменения, принадлежащие пользователю или сайту, вместо того, чтобы перезаписывать их.

2. Перейти к утвержденной новой версии исходного кода.

```
git checkout <approved-new-ref>
git rev-parse --short HEAD
```

Запишите целевую версию и сравните ее с утвержденной заявкой на изменение или примечанием к выпуску.

3. Обновите локальную среду Python в репозитории.

```
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

Используйте существующий `.venv`, если примечание к обновлению явно не требует его пересоздания. Записывайте ошибки зависимостей перед изменением базы данных.

4. Останавливается при неразрешенных ошибках источника или зависимостей.

Внимание:

Если при извлечении, установке зависимостей или активации виртуальной среды произошел сбой, не запускайте миграцию. Вернитесь к записанной предыдущей версии только в том случае, если база данных не была изменена, затем переустановите предыдущий набор зависимостей и повторно запустите дымовые проверки перед обновлением.

Извлечение и среда Python имеют утвержденную целевую версию, или обновление остановлено до внесения изменений в базу данных.

Запускайте миграцию базы данных и проверку документации только после успешного обновления зависимостей.

Выполнение миграций и проверок обновления исходного кода

Примените миграции базы данных и проверьте обновлённое развертывание BPM из исходного кода после смены ревизии.

Извлечение и зависимости были успешно обновлены, а свидетельства резервной копии базы данных были собраны, пока BPM был остановлен.

Обновления могут включать миграции базы данных. BPM документирует прямую миграцию с помощью `alembic upgrade head`; автоматический переход на более раннюю версию или безопасный откат схемы не гарантируются.

1. Примените миграцию базы данных к намеченной базе данных.

```
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

Используйте фактическое значение развертывания для `BPM_DATABASE_URL`. Немедленно остановитесь, если результаты миграции неясны или не удались.

2. Запустите обновлённый процесс BPM и проверьте его текущий интерфейс.

Проверьте работоспособность и готовность, затем откройте маршрут Библиотеки. Если документация требуется для развертывания, убедитесь, что `/help/` сообщает о готовом установленном артефакте, а не о его отсутствии или несовместимости.

3. Остановитесь при сбое миграции или проверки во время работы.

Внимание:

После выполнения миграции не считайте откат `Git checkout` достаточным. Не запускайте `alembic downgrade`, если проверенное примечание к выпуску прямо не подтверждает поддержку точной пары ревизий. При необходимости отката используйте резервную копию базы данных до обновления как безопасную точку восстановления.

Обновлённая ревизия исходного кода завершила миграции и проверку во время работы, либо обновление остановлено в указанной точке сбоя.

Запустите BPM и запустите тему проверки после обновления, прежде чем возвращать экземпляр пользователям или интеграторам.

Проверка обновления и применение условий остановки отката

Запустите обновленный процесс BPM, проверьте зонды, UI, доказательства API и состояние документации, затем решите, следует ли принять, остановить или восстановить из резервной копии.

Проверки миграции, дыма источников и восстановления документации завершены или зафиксирована точка сбоя.

Проверка после обновления подтверждает, что обновленный исходный код пригоден для использования в текущей контролируемой среде. Он не доказывает готовность к работе, поведение обратного прокси-сервера, HA, управляемое резервное копирование или принудительное выполнение Firefox во время выполнения.

1. Запустите обновленный исходный процесс и проверьте зонды.

```
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Ожидаемые ответы: {"status": "ok"} и {"status": "ready", "ready": true}.

2. Проверьте продукт UI, свидетельства API и статус документации.

Откройте <http://127.0.0.1:8000/profiles>, запустите необходимые примеры многократного использования API, при необходимости подтвердите экспортированное свидетельство `policies.json` и убедитесь, что `/help/` не сообщает об устаревшей, несовместимой или отсутствующей документации для установленного артефакта.

3. Принимайте обновление только после того, как записаны все необходимые проверки.

Запишите предыдущую версию, целевую версию, результат миграции, результат исходного дыма, результат проверки/сборки документации, ответы о работоспособности/готовности, результат дыма UI, доказательства API/экспорта и любые известные элементы управления отложенным производством.

4. Применяйте условия отката или остановки, не изобретая автоматического восстановления.

Если перед миграцией произошел сбой, вернитесь к предыдущей исходной версии и переустановите зависимости. Если после миграции произошел сбой, остановите BPM, восстановите проверенную резервную копию базы данных перед обновлением, вернитесь к предыдущей исходной версии, переустановите зависимости и повторите дымовые проверки работоспособности/UI. Если данные резервной копии отсутствуют, восстановление не проверено или данные изменились после миграции, остановитесь и перейдите к эскалации вместо заявления об успешном откате.

Внимание:

Не помечайте обновление как завершенное, если проверка не удалась, документация устарела, результаты экспорта неожиданно отличаются, миграция неопределенна или свидетельства отката неполны.

Обновление либо принимается с полными доказательствами, останавливается на известном блокировщике, либо восстанавливается из резервной копии перед обновлением с явным доказательством.

Используйте полученные данные в записке о передаче операций и при планировании будущего производства/НА/обратного прокси.

Проверка работоспособности и готовности BPM

Используйте текущие проверки работоспособности и готовности как упрощённую предварительную проверку состояния перед рабочими процессами API.

Установить `$BPM_BASE_URL`. Для настройки хоста, запуска служб, журналов, обновлений из источника и сведений об операционной системе используйте Руководство администратора по развёртыванию источника; в этом разделе API описана только проверка интеграции.

Эта задача охватывает API-HEALTH-001 и API-HEALTH-002. BPM предоставляет проверки без префиксов в `/health` и `/health/ready`. BPM не предоставляет диагностику зависимостей посредством этих проверок; они не имеют версии и не заменяют политику повторных попыток или инцидентов для конкретного продукта.

1. Вызовите проверку работоспособности перед запуском короткой последовательности API.

Пример запроса: `GET $BPM_BASE_URL/health`. Пример ответа: `200 {"status": "ok"}`. Пример ошибки: в данный момент не объявлено ни одной ошибки приложения; Сбои транспорта, шлюза или процесса обрабатываются вне BPM.

2. Вызовите проверку готовности перед выполнением операций, зависящих от завершения запуска приложения.

Пример запроса: `GET $BPM_BASE_URL/health/ready`. Пример ответа: `200 {"status": "ready", "ready": true}`. Пример ошибки: в настоящее время не отображается тело ухудшенного состояния или сведения о зависимостях.

3. Используйте готовность как шлюз, а не как полную инвентаризацию зависимостей.

Если `ready` равен `true`, интегрирующий продукт может продолжить обнаружение, проверку, импорт, экспорт или вызовы жизненного цикла профиля. Из этого ответа он не должен выводить сведения о работоспособности базы данных, сведения о кэше схемы, состояние портала документации, состояние очереди или топологию развёртывания.

4. Продолжайте опрос и повторяйте принятие решений в интегрируемом продукте.

BPM не объявляет интервал опроса, заголовок повтора после, цель уровня обслуживания, контракт автоматического выключателя, запрос ID или веб-перехватчик для изменений зонда. Сохраните свою собственную временную метку, наблюдаемый статус, тайм-аут и решение о повторной попытке.

5. Отделите проверки установления связи API от устранения неполадок при развёртывании источника.

Если зонды недоступны или ответ отличается от примеров, переключитесь на Руководство администратора по развёртыванию исходного кода для проверки процессов, среды, базы данных, журнала и обновлений из источника вместо того, чтобы расширять эту задачу интеграции API до инструкций по развёртыванию.

6. Запишите результат оперативного установления связи с помощью действия API нисходящего потока.

Внимание:

Успешный ответ о работоспособности или готовности не аутентифицирует вызывающих абонентов, не авторизует операции, не резервирует версии профиля, не гарантирует последующий успех API и не доказывает соблюдение требований времени выполнения Firefox. Относитесь к этому как к одному небольшому предполетному сигналу.

Интегрирующий продукт имеет минимальное и актуальное оперативное подтверждение BPM для обеспечения работоспособности и готовности без зависимости от недокументированного поведения при развертывании.

После успешной проверки готовности продолжите обнаружение OpenAPI или конкретную тему рабочего процесса API для проверки, импорта/экспорта или операций жизненного цикла профиля.

Интеграция через API BPM

Аудитория интеграции и границы API

Раздел об интеграции API в руководстве администратора и DevOps предназначен для внешних инструментов, которые используют текущий программный API BPM без гарантий неподдерживаемых возможностей.

Аудитория

Используйте это руководство при создании или проверке продукта конфигурации, соответствия требованиям, оркестрации, инвентаризации или контроля, который обменивается данными профиля и политики Firefox с BPM. Основными читателями являются интеграторы API, проверяющие безопасность, инженеры по выпуску, которым нужен повторяемый контракт для синхронизации профилей, шлюзов проверки, импорта/экспорта политик, метаданных соответствия, инвентаризации и проверок работоспособности.

Граница доказательств

Руководство основано на сгенерированном контракте OpenAPI, поддерживаемом инвентаре API, источнике маршрута/модели и целевых тестах API. В нем документируется API BPM, который уже представлен в версии 0.9.1. Он не разрешает новые конечные точки, модели безопасности, схемы данных, семантику повторных попыток или обещания совместимости.

Поддерживаемые роли потребителей

Поддерживаемые потребители — это общие роли: менеджер конфигурации, сканер соответствия, шлюз проверки политики, сборщик инвентаризации профилей, оркестратор развертывания и операционный монитор. Они могут вызывать операции службы, профиля, импорта/экспорта Firefox, проверки и работоспособности, перечисленные в инвентаре API.

Неподтвержденные претензии

Не описывайте соединитель для конкретного продукта, интеграцию с торговой площадкой, сертифицированную интеграцию, партнерство, управляемый агент или рабочий процесс поставщика, если этот соединитель фактически не поставляется. Маршруты HTML, статические ресурсы, сгенерированная схема интерфейсы и вспомогательные средства Python с префиксом подчеркивания не являются стабильной машинной интеграцией API.

Право собственности на данные

ВРМ владеет нормализованной моделью профиля, состоянием проверки, версией, состоянием жизненного цикла, непрозрачным объектом соответствия и каноническим экспортным рендерингом Firefox. Интегрируемые продукты управляют своим планированием, авторизацией, контрольным журналом, политикой повторных попыток, записью исключений, решением о развертывании и любой проверкой времени выполнения за пределами ВРМ.

Поддерживаемые шаблоны интеграции

BPM поддерживает общие шаблоны API для рабочих процессов профиля, проверки, документов Firefox, метаданных, инвентаризации и работоспособности.

Синхронизация профилей

Интегрирующий продукт может составлять список профилей, читать профиль с помощью ID, создавать нормализованный профиль, исправлять изменяемые поля профиля, архивировать, восстанавливать, окончательно удалять или сбрасывать библиотеку. Синхронизация должна рассматривать профиль ID и версию как состояние, принадлежащее BPM, и не должна предполагать массовые операции, изменение имени или автоматический откат для нескольких запросов.

Ворота проверки

Шлюз проверки может отправить кандидатный канонический документ `policies.json` в поддерживаемую конечную точку проверки до того, как рабочий процесс развертывания примет его. Шлюз должен проверять тело ответа, а также код состояния, поскольку проверка может вернуть успешный статус HTTP с помощью `ok=false`.

Импорт и экспорт политик

Интеграция позволяет импортировать Firefox `policies.json` как профиль BPM и экспортировать сохраненный профиль BPM обратно в канонический JSON Firefox. Не заменяйте нормализованную модель профиля канонической формой документа Firefox; импорт/экспорт — это граница между этими представлениями.

Передача метаданных соответствия

API предоставляет непрозрачный объект `compliance` при операциях создания, обновления и импорта профиля. Интеграции могут передавать свои собственные метаданные через BPM, но BPM не публикует версию для этого объекта и не проверяет семантику внешнего соответствия.

Инвентаризация и проверки работоспособности

Потребители в стиле инвентаризации могут перечислять профили, запрашивать статистику для одного и того же контекста фильтра, читать отдельные профили, а также вызывать конечные точки обслуживания, работоспособности и готовности. Ответы о работоспособности намеренно компактны и не раскрывают диагностику каждой зависимости, состояние документации или готовность к развертыванию за пределами текущих фиксированных полей.

Неподдерживаемые шаблоны

В руководстве не документируются аутентификация, авторизация, ограничения скорости, ключи идемпотентности, запрос идентификаторы, веб-перехватчики, массовый экспорт, массовый CRUD, нумерация страниц курсора, коннекторы для конкретных продуктов, партнерские отношения с поставщиками или оркестровка транзакций, поскольку BPM не предоставляет этих гарантий.

Соглашения API

Используйте эти соглашения при чтении контрактов OpenAPI BPM и построении запросов на интеграцию.

База URL-адреса и обнаружение

В примерах `$BPM_BASE_URL` используется в качестве заполнителя развертывания. Узнайте текущий контракт от `/openapi.json` и, только для интерактивной проверки, схему FastAPI интерфейсы в `/docs` и `/redoc`. Портал документации будущего продукта зарезервирован для `/help/`, поэтому не рассматривайте `/docs` как маршрут документации DITA.

Типы контента

Большинство программных операций используют тела запросов и ответов JSON. Импорт Firefox принимает `application/json` и `multipart/form-data`; неподдерживаемые типы мультимедиа могут возвращать 415. Экспорт Firefox возвращает `application/json` и может добавить `Content-Disposition`, когда запрашивается вывод загрузки.

Идентификаторы и жизненный цикл

Профиль идентификаторы — это целочисленные идентификаторы пути, принадлежащие BPM. Сохраните возвращенные `id` и `revision` после операций создания, импорта, чтения и обновления. Имена профилей уникальны и не могут быть изменены через `ProfileUpdate`. Значения жизненного цикла активны, заархивированы и все такое; `include_deleted` — это флаг совместимости, который необходимо интерпретировать вместе с жизненным циклом.

Модели и каналы схемы

`ProfileCreate`, `ProfileUpdate` и `ProfileRead` — это нормализованные формы профиля BPM. Импорт/экспорт Firefox использует канонический `policies.json` с `policies` верхнего уровня. При проверке используется `ValidationRequest` и поддерживаемые каналы схемы, такие как Firefox ESR и Release; простое сопоставление политики остается совместимым поведением, а не предпочтительной внешней формой.

Фильтрация, сортировка и нумерация страниц

Операции со списком профилей и статистикой поддерживают фильтры поиска, схемы, состояния проверки, жизненного цикла и включения-удаления. Пагинация списка смещена/ограничена по умолчанию `limit=50`, максимальным значением `200` и `offset=0`. При сортировке используются текущие описанные поля, такие как `created_at`, `updated_at`, `name`, `schema_version` и `id` с порядком `asc` или `desc`.

Статус и обработка ошибок

Используйте коды состояния и форму тела вместе. 200, 201 и 204 являются успешными семействами, но проверка может вернуть 200 с помощью `ok=false`. В телах ошибок используется FastAPI detail и может содержать строку, список проверки или объект BPM с `message`, `error` или `issues`.

Текущие ограничения API и ограничения безопасности

BPM четко документирует текущие ограничения, поэтому интеграция не подразумевает гарантий, которые не предоставляются.

Ограничения безопасности

Текущий API не имеет уровня аутентификации или авторизации. По умолчанию для CORS используется источник *, если только настройки развертывания не переопределяют его. Документация не должна подразумевать аутентификацию токена API, OAuth, управление доступом на основе ролей, изоляцию арендатора или поведение авторизации в рабочей среде.

Управление версиями и совместимость

API не имеет префикса версии, политики совместимости, заголовка устаревания или версии типа носителя. Интеграции должны закреплять версию BPM, с которой они тестируются, и следить за `/openapi.json`, инвентарем API и примечаниями к выпуску, а не предполагать гарантированную обратную совместимость.

Параллелизм, идемпотентность и повторная попытка

Только PATCH поддерживает оптимистический параллелизм посредством необязательного `expected_revision`. Создание, импорт, удаление, восстановление, сброс и экспорт не раскрывают ETags, заголовки предусловий, ключи идемпотентности или контракты повтора. Потребители должны разработать свои собственные безопасные повторные попытки и поведение восстановления повторяющихся имен.

Массовые операции и границы транзакций

BPM не предоставляет ни конечной точки массового профиля CRUD, ни конечной точки массового экспорта, ни разбиения на страницы курсора, ни контракта многооперационной транзакции. Неудачная многоэтапная интеграция не откатывается автоматически BPM; интегрирующий продукт владеет записями последовательности восстановления и аудита.

Ограничения конверта ошибок

В ответах на ошибки не используется один стабильный конверт с версией. Статусы 400, 404, 409, 415, 422 и 503 могут иметь разные формы `detail`. Потребители должны разделяться по функциям, статусу и документально подтвержденной форме тела.

Исключенные поверхности и поверхности HTML

Шесть операций OpenAPI web представляют собой маршруты продуктов HTML, а не стабильную машинную интеграцию API. `/i18n/{locale}.json`, `/favicon.ico`, `/static/*`, сгенерированная схема интерфейсы и вспомогательные функции Python с префиксом подчеркивания исключены из тем, посвященных операциям интеграции.

Пределы реагирования на состояние здоровья

`/health` и `/health/ready` предоставляют только компактные поля состояния. Готовность не сообщает о диагностике каждой зависимости, сведениях о базе данных, работоспособности схемы, состоянии портала документации или утверждении развертывания. Операционные мониторы должны рассматривать их как минимальные конечные точки установления связи.

Повторное использование примеров API и шаблонов среды DevOps

Используйте независимые от языка шаблоны запросов, шаблоны среды, копируемые команды curl, составные примеры и компактные утверждения Python для текущих рабочих процессов BPM API.

Установите BPM_BASE_URL для тестируемого экземпляра BPM. Не помещайте учетные данные, имена клиентов, личные имена хостов, частные хосты или рабочий профиль идентификаторы в примеры многократного использования; BPM в настоящее время не документирует аутентификацию API.

Эта задача охватывает API-HEALTH-001, API-HEALTH-002, API-PROFILE-001, API-PROFILE-003, API-PROFILE-004, API-PROFILE-005, API-FF-001, API-VAL-001 и API-FF-002. В примерах намеренно используются \$BPM_BASE_URL, синтетические имена, release-153, утверждения ответа и одноразовое приспособление идентификаторы вместо хостов или секретов, специфичных для среды.

1. Прежде чем копировать запросы на запуск, создайте шаблон среды многократного использования.

Пример запроса: `export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"`. Пример ответа: оболочка сохраняет один базовый URL-адрес, канал схемы, имя синтетического профиля и необязательный путь импорта для всего запуска. Пример ошибки: оболочка завершает выполнение присваивания, когда необходимое значение отсутствует.

```
# .env.devops-example.template
BPM_BASE_URL=
BPM_SCHEMA_CHANNEL=release-153
BPM_PROFILE_NAME=docs-api-example
BPM_JSON_IMPORT_PATH=./policies.json

export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"
export BPM_SCHEMA_CHANNEL="${BPM_SCHEMA_CHANNEL:-release-153}"
export BPM_PROFILE_NAME="${BPM_PROFILE_NAME:-docs-api-example}"
export BPM_JSON_IMPORT_PATH="${BPM_JSON_IMPORT_PATH:-./policies.json}"
```

2. Контролируйте каждую повторно используемую последовательность с помощью зондов работоспособности и готовности.

Пример запроса: `curl -fsS "$BPM_BASE_URL/health"` и `curl -fsS "$BPM_BASE_URL/health/ready"`. Пример ответа: `{"status": "ok"}` и `{"status": "ready", "ready": true}`. Пример ошибки: curl завершает работу с ненулевым значением для ошибок транспорта или ответов, отличных от 2xx, при использовании `-f`.

```
curl -fsS "$BPM_BASE_URL/health"
curl -fsS "$BPM_BASE_URL/health/ready"
```

3. Повторно используйте фрагменты жизненного цикла профиля для создания, просмотра, чтения и защищенного обновления.

Пример запроса: POST \$BPM_BASE_URL/api/profiles, GET \$BPM_BASE_URL/api/profiles, GET \$BPM_BASE_URL/api/profiles/\$PROFILE_ID и PATCH \$BPM_BASE_URL/api/profiles/\$PROFILE_ID. Пример ответа: 201 ProfileRead, 200 ProfileRead[], 200 ProfileRead и увеличенный revision. Примеры ошибок: 400, 404, 409 устаревший expected_revision или 422.

```
PROFILE_JSON="$(curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-api-example","schema_version":"release-153","flags":{"DisableTelemetry":true}}' \
"$BPM_BASE_URL/api/profiles")"
PROFILE_ID="$(python -c 'import json,sys; print(json.load(sys.stdin)["id"])' <<<"$PROFILE_JSON")"
PROFILE_REVISION="$(python -c 'import json,sys; print(json.load(sys.stdin)["revision"])'
<<<"$PROFILE_JSON")"

curl -fsS "$BPM_BASE_URL/api/profiles?q=docs-api-example&lifecycle=active&limit=50&offset=0"
curl -fsS "$BPM_BASE_URL/api/profiles/$PROFILE_ID"
curl -fsS -X PATCH -H 'Content-Type: application/json' \
-d '{"flags":
{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":'$PROFILE_REVISION'" \
"$BPM_BASE_URL/api/profiles/$PROFILE_ID"
```

4. Повторно используйте фрагменты проверки и импорта JSON с явной формой полезных данных.

Пример запроса: POST \$BPM_BASE_URL/api/validate/release-153 и POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json. Пример ответа: 200 {"ok":true,"profile":"release-153"} и 201 ProfileRead. Примеры ошибок: 400, 409, 415, 422 или 503.

```
curl -fsS -H 'Content-Type: application/json' \
-d '{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}' \
"$BPM_BASE_URL/api/validate/$BPM_SCHEMA_CHANNEL"

curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-json-import-example","schema_version":"release-153","compliance":{"source":"docs-devops-template"},"document":{"policies":{"DisableTelemetry":true}}}' \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"
```

5. Повторное использование составных фрагментов импорта и экспорта для передачи обслуживания на основе файлов.

Пример запроса: загрузка multipart/form-data в POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json, а затем GET \$BPM_BASE_URL/api/export/profiles/\$PROFILE_ID/firefox/policies.json?pretty=1. Пример ответа: 201 ProfileRead для импорта и 200 application/json, содержащий канонический Firefox, policies для экспорта. Пример ошибки: 404 отсутствует профиль или 422 недействительный профиль ID.

```
curl -fsS \
```

```
-F "name=docs-multipart-import-example" \
-F "schema_version=$BPM_SCHEMA_CHANNEL" \
-F 'compliance={"source":"docs-devops-template"}' \
-F "file=@$BPM_JSON_IMPORT_PATH;type=application/json" \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"

curl -fsS "$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1"
```

6. Используйте компактный помощник Python, если для интеграции необходимы многократно используемые утверждения ответа.

Пример запроса: `session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)`. Пример ответа: помощник немедленно вызывает вызов, если только статус и тело JSON не соответствуют утверждению. Пример ошибки: сбой утверждений, исключения тайм-аута, устаревшие ошибки `expected_revision` или структурированные API должны регистрироваться интегрирующим продуктом.

```
import os
import requests

base_url = os.environ["BPM_BASE_URL"].rstrip("/")
schema_channel = os.environ.get("BPM_SCHEMA_CHANNEL", "release-153")
session = requests.Session()

ready = session.get(f"{base_url}/health/ready", timeout=10)
ready.raise_for_status()
assert ready.json() == {"status": "ready", "ready": True}

payload = {"document": {"policies": {"DisableTelemetry": True, "BlockAboutConfig": True}}}
validation = session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)
validation.raise_for_status()
assert validation.json() == {"ok": True, "profile": schema_channel}

created = session.post(
    f"{base_url}/api/profiles",
    json={"name": "docs-python-example", "schema_version": schema_channel, "flags":
payload["document"]["policies"]},
    timeout=10,
)
created.raise_for_status()
profile = created.json()
assert profile["schema_version"] == schema_channel

exported = session.get(
    f"{base_url}/api/export/profiles/{profile['id']}/firefox/policies.json?pretty=1",
    timeout=10,
)
exported.raise_for_status()
assert exported.json() == {"policies": payload["document"]["policies"]}
```

Внимание:

Сохраняйте статус ответа, профиль ID, версию, канал схемы, хэш экспортированного документа и результат утверждения во внешних доказательствах. Не копируйте реальные имена хостов, секреты, ключи API, значения секретов или производственные идентификаторы в примеры документации.

Интегрирующий продукт содержит многократно примеры curl и Python, которые можно безопасно копировать, они не зависят от хостов развертывания, синхронизируются с текущим поведением API BPM и привязаны к явным утверждениям ответа.

Объедините эти примеры с модулями выполнения «извлечение/список/чтение», «проверка перед применением», «импорт-проверка-экспорт» и «аудит отказов» при построении полных внешних рабочих процессов управления продуктом.

Синхронизация данных жизненного цикла профиля

Используйте список профилей, статистику, читайте, создавайте и обновляйте конечные точки, не прибегая к массовому или транзакционному поведению.

Установите `$BPM_BASE_URL` и прочитайте `/openapi.json` для текущих фигур `ProfileCreate`, `ProfileUpdate` и `ProfileRead`.

Эта задача охватывает API-PROFILE-001, API-PROFILE-002, API-PROFILE-003, API-PROFILE-004 и API-PROFILE-005. Магазин вернул `id` и `revision`; не делайте выводов о личности только по имени.

1. Перечислите профили с фильтрами, прежде чем выбирать профиль для синхронизации.

Пример запроса: `GET $BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Пример ответа: `200 ProfileRead[]`. Пример ошибки: 422 для недопустимого ввода запроса или ранжированного запроса.

2. Прочитайте статистику профиля для того же контекста фильтра, когда вам нужны подсчеты.

Пример запроса: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`. Пример ответа: `200 {"filtered":1,"total":3}`. Пример ошибки: 422 из-за неверного ввода запроса.

3. Читать выбранный профиль по целому числу ID и включать архивные записи только при необходимости.

Пример запроса: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Пример ответа: `200 ProfileRead` с `id`, `revision`, метками времени, жизненным циклом и состоянием проверки. Пример ошибки: 404 для отсутствующего или невидимого профиля; 422 для неверного пути или ввода запроса.

4. Создавайте нормализованный профиль BPM только в том случае, если интеграция владеет новым именем профиля.

Пример запроса: `POST $BPM_BASE_URL/api/profiles` с JSON `{"name":"corp-baseline","schema_version":"release-153","flags":{},"compliance":{}}`. Пример ответа: `201 ProfileRead`. Примеры ошибок: 400 недопустимая схема/профиль, 409 повторяющееся имя или 422 запрос или проверка политики.

5. Обновите изменяемые поля с помощью PATCH и отправьте `expected_revision`, когда вам нужен оптимистичный параллелизм.

Пример запроса: `PATCH $BPM_BASE_URL/api/profiles/42` с JSON `{"description":"Reviewed by control plane","expected_revision":7}`. Пример ответа: `200 ProfileRead` с увеличенной редакцией после обновления материала. Примеры ошибок: 400 недействительная схема/профиль, 404 отсутствующий или заархивированный профиль, 409 устаревший `expected_revision` или ошибка проверки 422.

6. Запишите синхронизированный профиль ID, версию, состояние проверки, канал схемы и все непрозрачные метаданные соответствия в интегрируемом продукте.

Внимание:

ВРМ не обеспечивает транзакцию с несколькими операциями. Если на более позднем этапе произойдет сбой, решения о восстановлении, аудите и повторных попытках принимает интегрирующий продукт.

Интеграция имеет воспроизводимый путь синхронизации профилей для операций списка, статистики, чтения, создания и обновления.

Используйте задачу разрушительного жизненного цикла перед операциями архивирования, восстановления, окончательного удаления или сброса.

Архивирование, восстановление, безвозвратное удаление и сброс профилей

Используйте деструктивные конечные точки жизненного цикла профиля с явными записями проверки и восстановления.

Прежде чем вызывать деструктивные конечные точки, подтвердите целевой профиль ID, текущее состояние жизненного цикла и запись внешнего утверждения.

Эта задача охватывает API-PROFILE-006, API-PROFILE-007, API-PROFILE-008 и API-PROFILE-009. В BPM нет токена массового подтверждения, нет конечной точки отмены для окончательного удаления и нет отката транзакций между операциями.

1. Архивируйте профиль, если вам нужно удалить его из активных рабочих процессов, сохранив при этом для просмотра.

Внимание:

Архивирование — это разрушительное изменение жизненного цикла активных рабочих процессов. Перед звонком запишите причину одобрения и профиль ID.

Пример запроса: `DELETE $BPM_BASE_URL/api/profiles/42`. Пример ответа: 204 без тела JSON. Примеры ошибок: 404 отсутствует профиль или 422 введен неверный путь.

2. Восстанавливайте заархивированный профиль только после проверки того, действительны ли имя и назначение профиля.

Пример запроса: `POST $BPM_BASE_URL/api/profiles/42/restore`. Пример ответа: 200 ProfileRead. Примеры ошибок: Профиль 404 не может быть восстановлен или найден; 422 введен неверный путь.

3. Окончательно удалять профиль только в том случае, если внешняя политика хранения разрешает удаление.

Внимание:

Постоянное удаление удаляет активный или заархивированный профиль и не имеет конечной точки отмены BPM. Экспортируйте или запишите необходимые доказательства перед звонком.

Пример запроса: `DELETE $BPM_BASE_URL/api/profiles/42/hard`. Пример ответа: 204 без тела JSON. Примеры ошибок: 404 отсутствует профиль или 422 введен неверный путь.

4. Сбрасывайте библиотеку только в одноразовых или явно одобренных средах.

Внимание:

При сбросе безвозвратно удаляются все профили в библиотеке, и в настоящее время токен подтверждения не требуется. Никогда не используйте его в качестве ярлыка для обычной синхронизации.

Пример запроса: DELETE \$BPM_BASE_URL/api/profiles/reset. Пример ответа: 200 {"deleted":3}. Поведение при ошибке: в настоящее время не объявлено ни одной ошибки приложения; сбои базы данных используют обработку инфраструктуры.

5. После любого деструктивного вызова заново перечислите профили и прочитайте затронутую запись, если это применимо.

Пример запроса: GET \$BPM_BASE_URL/api/profiles?lifecycle=all. Пример ответа: 200 ProfileRead[]. Пример ошибки: 422 неверный ввод запроса. Используйте эту проверку для обновления состояния запасов интегрируемого продукта.

6. Запишите запись внешнего аудита с операцией ID, методом HTTP, путем, профилем ID, утверждающим, причиной, меткой времени, статусом ответа и решением о восстановлении.

Ожидаемый результат: последующий проверяющий сможет различать действия архивирования, восстановления, полного удаления и сброса и сможет понять, почему интеграция считает эту операцию безопасной.

Деструктивные операции жизненного цикла профиля документируются с указанием запросов, ответов, ошибок, предупреждений и доказательств восстановления.

Если запись об утверждении отсутствует, остановите рабочий процесс интеграции перед вызовом окончательного удаления или сброса.

Импорт Firefox policies.json через API

Создайте нормализованный профиль BPM из канонического документа Firefox Enterprise policies.json, используя JSON или составной ввод.

Установите \$BPM_BASE_URL, выберите поддерживаемый канал схемы, например release-153, и подготовьте полный документ Firefox с объектом policies верхнего уровня.

Эта задача охватывает API-FF-001. Импорт принимает application/json с FirefoxPoliciesJsonImportRequest или multipart/form-data с полем file. BPM проверяет канонический документ Firefox, нормализует policies в профиль flags и сохраняет необязательные непрозрачные метаданные compliance в профиле BPM.

1. Отправьте запрос на импорт JSON, когда интегрирующий продукт уже имеет документ в памяти.

Пример запроса: POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json с Content-Type: application/json и JSON {"name":"api-json-import","description":"Imported by control product","schema_version":"release-153","compliance":{"framework":"cis","layer":"cis_l1"},"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}.

2. Используйте многочастный импорт, когда исходная система передает файл.

Пример запроса: POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json с multipart/form-data, полями name=api-multipart-import, schema_version=release-153, необязательным description, необязательным compliance={"framework":"cis","layer":"cis_l2"} в виде строки JSON и file=@policies.json;type=application/json. Среда выполнения также принимает составное поле document, но составная схема OpenAPI в настоящее время объявляет file; используйте file для новых примеров.

3. Прочитайте созданный профиль BPM из ответа 201 ProfileRead.

Пример ответа: 201 ProfileRead с schema_version="release-153", flags.DisableTelemetry=true, flags.BlockAboutConfig=true, сохраненными compliance, id и revision. Сохраните возвращенный профиль ID вместо того, чтобы определять личность по имени профиля.

4. Рассматривайте ошибки импорта как ошибки создания профиля и не допускайте частичного состояния профиля.

Примеры ошибок: 400 для неправильного формата JSON, неправильной формы документа или неподдерживаемого канала схемы; 409 для повторяющегося имени профиля; 415 для неподдерживаемого типа носителя; 422 в случае ошибок запроса или проверки политики Firefox.

5. Разделяйте канонические формы Firefox и нормализованные формы BPM в коде интеграции.

Запрос document.policies является границей развертывания Firefox. flags созданного профиля представляет собой нормализованную модель BPM. Не отправляйте объект flags

верхнего уровня в эту конечную точку импорта.

6. Запишите доказательства импорта в интегрируемый продукт.

Внимание:

Импорт создает профиль BPM, но не разворачивает настройки Firefox, не подтверждает принудительное выполнение во время выполнения и не предоставляет ключ идемпотентности. Запишите хеш исходного документа, канал схемы, возвращенный профиль ID, версию, метаданные соответствия и решение о восстановлении повторяющегося имени на внешнем носителе.

Документ `Firefox policies.json` представлен как нормализованный профиль BPM, который можно просматривать, проверять, экспортировать и отслеживать с помощью ID.

Используйте задачу экспорта, чтобы преобразовать сохраненный профиль обратно в канонический JSON Firefox, прежде чем передавать его в инструменты развертывания.

Экспорт канонического Firefox policies.json через API

Отображение одного профиля BPM как документа развертывания Firefox Enterprise с дополнительными параметрами загрузки и печати.

Сохраните профиль BPM ID, полученный при создании или импорте. Если профиль заархивирован, явно решите, разрешено ли интеграции экспортировать его с помощью `include_deleted=true`.

Эта задача охватывает API-FF-002. Экспорт считывает нормализованный flags BPM и возвращает канонический JSON Firefox с объектом policies верхнего уровня. Метаданные профиля BPM и непрозрачные метаданные compliance не экспортируются в документ развертывания Firefox.

1. Экспортируйте активный профиль в канонический формат Firefox JSON.

Пример запроса: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json`.

Пример ответа: `200 application/json cJSON {"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}`.

2. Добавить режим загрузки, когда вызывающему абоненту требуется имя файла вложения.

Пример запроса: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1`. Пример ответа: `200 application/json` плюс `Content-Disposition: attachment; filename="profile-42-policies.json"`. Тело JSON — это тот же документ развертывания.

3. Выбирайте параметры форматирования сознательно.

Пример запроса: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` выбирает отступ в два пробела, если `indent` опущен. Пример запроса: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?indent=0` использует явный параметр отступа. Пример ошибки: `422` для недопустимых значений ограниченного запроса.

4. Экспортируйте заархивированный профиль только в том случае, если у внешнего рабочего процесса есть причина для проверки.

Пример запроса: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?include_deleted=true&download=1&pretty=1`. Пример ответа: `200 application/json`. Примеры ошибок: `404`, когда отсутствующий или заархивированный профиль не виден; `422` из-за неверного пути или ввода запроса.

5. Проверьте экспортированный документ перед его передачей в другую систему.

Пример запроса: `POST $BPM_BASE_URL/api/validate/release-153` с `cJSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Пример ответа: `200 {"ok":true,"profile":"release-153"}`. Проверка подтверждает форму схемы, а не успех развертывания среды выполнения Firefox.

6. Запишите точный экспортированный артефакт на внешний объект.

Внимание:

Экспорт не имеет ETag, заголовка предварительного условия, массовой конечной точки или оболочки транзакции. Запишите профиль ID, ревизию, наблюдаемую перед экспортом, канал схемы, параметры запроса, хэш экспортированного документа и решение о последующей передаче за пределы BPM.

Интеграция включает канонический артефакт `Firefox policies.json`, который исключает метаданные только для BPM и может быть предоставлен в инструменты развертывания.

Если проверка не удалась или версия профиля изменилась между чтением и экспортом, остановите передачу обслуживания и обновите состояние профиля.

Проверка кандидата Firefox policies.json через API

Используйте конечную точку проверки в качестве шлюза предварительной проверки для документов-кандидатов Firefox Enterprise policies.json и структурированной обработки ошибок.

Выберите целевой канал схемы, например release-153 или esr-140.13, и подготовьте тело ValidationRequest с полным документом Firefox в document.policies.

Эта задача охватывает API-VAL-001. BPM проверяет контент-кандидат policies.json перед импортом, передачей экспорта или внешним шлюзом проверки политики. Простое сопоставление политик по-прежнему допускается для совместимости, но в новых примерах интеграции используется полная форма Firefox policies.json.

1. Отправьте успешный запрос на проверку выбранного канала схемы.

Пример запроса: POST \$BPM_BASE_URL/api/validate/release-153 с JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}. Пример ответа: 200 {"ok":true,"profile":"release-153"}.

2. Рассматривать ok=false как неисправный шлюз, даже если статус HTTP равен 200.

Пример запроса: POST \$BPM_BASE_URL/api/validate/release-153 с JSON {"document":123}. Пример ответа: 200 {"ok":false,"profile":"release-153","detail":"Expected object with policy mappings","error":"Expected object with policy mappings"}. Проверьте тело ответа, а не только статус HTTP.

3. Обрабатывать некорректные канонические ошибки документов Firefox как ошибки формы запроса.

Пример запроса: POST \$BPM_BASE_URL/api/validate/release-153 с JSON {"document":{"policies":[]}}. Пример ошибки: 400, где detail.message="Firefox policies.json validation failed" и detail.issues[].path указывают на policies.

4. Отклонять неподдерживаемые каналы схемы, прежде чем доверять результатам проверки.

Пример запроса: POST \$BPM_BASE_URL/api/validate/beta-999 с JSON {"document":{"policies":{"DisableTelemetry":true}}}. Пример ошибки: 404 с подробным текстом Unknown profile 'beta-999'. Зарегистрированная, но недоступная схема возвращает 503.

5. Сопоставьте ошибки проверки политики с отзывами рецензентов, требующими принятия мер.

Пример запроса: POST \$BPM_BASE_URL/api/validate/release-153 с JSON {"document":{"policies":{"Proxy":{"Mode":"bogus"}}}}. Пример ошибки: 422 с detail.message="Policy validation failed" и полями выдачи policy, path и message.

6. Запишите решение о проверке в интегрируемом продукте.

Внимание:

Проверка — это только шлюз формы схемы и политики. Он не разворачивает Firefox, не проверяет соблюдение требований во время выполнения, не аутентифицирует вызывающих абонентов, не резервирует версию профиля и не предоставляет ключ идемпотентности. Сохраните канал, хеш-кандидат документа, статус ответа, значение ok, структурированные проблемы и решение рецензента во внешнем хранилище.

Интеграция может блокировать небезопасную передачу обслуживания кандидата `policies.json` и сохранять структурированные доказательства успешных и неудачных решений.

Если проверка прошла успешно, продолжите импорт, обновление профиля или экспорт передачи обслуживания, используя тот же канал схемы и записанный хеш-кандидат.

Сценарии жизненного цикла и восстановление после сбоев

Запуск сценария извлечения, сравнения и обновления контрольного продукта

Используйте существующий профиль и проверку API, чтобы получить состояние BPM, сравнить его снаружи и обновить один профиль с явным подтверждением владения и параллельного доступа.

Установите `$BPM_BASE_URL` и решите, какой внешний продукт управления владеет логикой сравнения, расчетом желаемого состояния, политикой повторных попыток и записями аудита. BPM не предоставляет API сравнения или оболочку транзакций.

В этом сценарии используются API-PROFILE-001, API-PROFILE-003, API-PROFILE-005 и API-VAL-001. BPM владеет сохраненным состоянием профиля и его проверкой. Интегрирующий продукт владеет желаемым состоянием, результатами сравнения, одобрением рецензента и решениями о повторных попытках.

1. Извлеките профили кандидатов из библиотеки с теми же правилами фильтрации, которые продукт управления использует для владения.

Пример запроса: `GET $BPM_BASE_URL/api/profiles?`

`q=corp&lifecycle=active&limit=50&offset=0`. Пример ответа: `200 ProfileRead[]`. Пример ошибки: `422` из-за неверного ввода запроса.

2. Прочитайте каждый выбранный профиль ID и сохраните его текущий `revision`.

Пример запроса: `GET $BPM_BASE_URL/api/profiles/42`. Пример ответа: `200 ProfileRead c flags, compliance, schema_version и revision`. Примеры ошибок: `404` профиль отсутствует или заархивирован; `422` введен неверный путь.

3. Сравните извлеченный профиль с внешним желаемым состоянием вне BPM.

Ожидаемый результат: интеграция продукта записывает измененные ключи политики, неизменные ключи, отсутствующие ключи, рецензента и источник желаемого состояния. Не предполагайте, что существует конечная точка сравнения BPM, соединитель торговой площадки или сертифицированный рабочий процесс.

4. Перед применением обновления проверьте потенциальный документ Firefox.

Пример запроса: `POST $BPM_BASE_URL/api/validate/release-153 c JSON {"document": {"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`. Пример ответа: `200 {"ok":true,"profile":"release-153"}`. Примеры ошибок: `400`, `404`, `422` или `503` в зависимости от формы документа, канала, действительности политики и доступности схемы.

5. Исправьте профиль BPM только той версией, которая наблюдалась до сравнения.

Пример запроса: `PATCH $BPM_BASE_URL/api/profiles/42 c JSON {"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":`

`{"source": "control-product", "decision": "approved"}}`. Пример ответа: 200 ProfileRead с увеличенным revision. Примеры ошибок: 400, 404, 409 устаревший expected_revision или 422.

6. Обработка сбоев без автоматического повтора.

Внимание:

Если проверка не удалась, редакция устарела или результат обновления не соответствует желаемому состоянию, остановите рабочий процесс, перечитайте состояние BPM и создайте запись внешнего аудита. BPM не предоставляет транзакцию с несколькими операциями, автоматический откат, ключ идемпотентности или контракт безопасного повтора.

Продукт управления имеет исполняемый поток извлечения/сравнения/обновления с явным владением государством, доказательствами проверки, защитой от изменений и границами восстановления после сбоя.

Если обновленный профиль необходимо развернуть в другом месте, продолжите задачу экспорта Firefox и сохраните профиль ID, версию, ответ проверки и запись утверждения.

Запуск сценария импорта, проверки, экспорта и передачи соответствия требованиям

Используйте существующие функции работоспособности, импорта, проверки, профиля и экспорта API, чтобы передать проверенный документ Firefox с доказательствами соответствия.

Установите `$BPM_BASE_URL`, выберите канал схемы и решите, какой системе принадлежит исходный документ, результат сканирования на соответствие, одобрение рецензента, экспортированный артефакт и решение о восстановлении.

В этом сценарии используются API-HEALTH-001, API-HEALTH-002, API-FF-001, API-PROFILE-003, API-VAL-001 и API-FF-002. BPM хранит нормализованный профиль и непрозрачные метаданные соответствия; интегрирующий продукт отвечает за интерпретацию сканирования и последующее развертывание.

1. Перед передачей обслуживания начните с проверки состояния здоровья и готовности.

Примеры запросов: GET `$BPM_BASE_URL/health` и GET `$BPM_BASE_URL/health/ready`. Примеры ответов: 200 `{"status": "ok"}` и 200 `{"status": "ready", "ready": true}`. Пример ошибки: сбой транспорта или шлюза обрабатывается вне BPM, поскольку в данный момент ошибка приложения не объявлена.

2. Импортируйте проверенный документ Firefox с метаданными соответствия, прикрепленными к профилю BPM.

Пример запроса: POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json` с JSON `{"name": "reviewed-corp-baseline", "schema_version": "release-153", "compliance": {"scanner": "external-control", "result": "passed", "ticket": "SEC-42"}, "document": {"policies": {"DisableTelemetry": true, "BlockAboutConfig": true}}}`. Пример ответа: 201 ProfileRead. Примеры ошибок: 400, 409, 415 или 422.

3. Прочтите созданный профиль и запишите поля владения BPM отдельно от внешних свидетельств.

Пример запроса: GET `$BPM_BASE_URL/api/profiles/42`. Пример ответа: 200 ProfileRead с `id`, `revision`, `flags`, `schema_version` и непрозрачным `compliance`. Примеры ошибок: 404 или 422.

4. Подтвердите канонический документ Firefox, который будет передан.

Пример запроса: POST `$BPM_BASE_URL/api/validate/release-153` с JSON `{"document": {"policies": {"DisableTelemetry": true, "BlockAboutConfig": true}}}`. Пример ответа: 200 `{"ok": true, "profile": "release-153"}`. Примеры ошибок: 400, 404, 422 или 503.

5. Экспортируйте профиль BPM как канонический документ развертывания Firefox.

Пример запроса: GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Пример ответа: 200 application/json с Content-Disposition и policies верхнего уровня. Метаданные профиля BPM и метаданные соответствия не экспортируются.

6. Остановиться при несоответствии и записать запись восстановления после сбоя.

Внимание:

Если доказательства импорта, проверки, обратного считывания или экспорта отличаются от утвержденного исходного документа, не повторяйте попытку вслепую. Сохраните хеш источника, профиль ID, версию, тело проверки, хэш экспорта, результат сканирования, рецензента, статус ответа и решение о восстановлении за пределами BPM.

Интегрирующий продукт имеет исполняемый поток импорта-проверки-экспорта с передачей соответствия требованиям, каноническим экспортом и явными доказательствами восстановления.

Используйте инструменты развертывания вне BPM, чтобы применить экспортированный `policies.json`; Проверка BPM не подтверждает соблюдение требований времени выполнения Firefox.

Получение и просмотр профилей BPM из внешнего продукта управления

Создайте сторону извлечения/списка/чтения в интеграции управляющего продукта с явным владением, фильтрами и границами доказательств.

Установить `$BPM_BASE_URL`. Решите, какой продукт внешнего контроля владеет выбором инвентаря, хранилищем для сравнения, политикой повторных попыток и доказательствами аудита; BPM владеет только записями профилей, возвращаемыми его API.

В этом рабочем процессе используются API-PROFILE-001, API-PROFILE-002 и API-PROFILE-003. Это безопасная точка входа, доступная только для чтения, перед рабочими процессами сравнения/обновления, проверки перед применением, импорта-проверки-экспорта или передачи метаданных о соответствии требованиям.

1. Подтвердите область действия управляющего продукта перед чтением состояния BPM.

Запишите внешнего владельца, цель, канал схемы, правила фильтрации и сведения о том, исключены ли архивные профили. Используйте `lifecycle=active`, если процедура восстановления явно не требует `include_deleted=true`.

2. Профили списка с детерминированными фильтрами и пейджингом.

Пример запроса: `GET $BPM_BASE_URL/api/profiles?`

`q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`. Пример ответа: `200 ProfileRead[]`. Пример ошибки: 422 для недопустимых параметров запроса.

3. Собирайте дополнительные данные инвентаризации отдельно от данных профиля.

Пример запроса: `GET $BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`. Пример ответа: `200 application/json`. Свидетельство количества магазинов как рекомендательный контекст инвентаризации, а не как блокировка или граница транзакции.

4. Перед сравнением прочтите каждый выбранный профиль пользователем ID.

Пример запроса: `GET $BPM_BASE_URL/api/profiles/42?include_deleted=false`. Пример ответа: `200 ProfileRead c id, revision, schema_version, flags и compliance`. Примеры ошибок: 404 или 422.

5. Сохранение снимка чтения за пределами BPM для управляющего продукта.

Профиль хранилища ID, редакция, канал схемы, запрос фильтра, временная метка чтения, исходный URL-адрес, выбранные ключи политики и запуск управляющего продукта ID. Не храните секреты или частные имена хостов в примерах общей документации.

6. Остановиться, если результату запроса нельзя доверять.

Внимание:

Если данные разбиения по страницам, фильтрации или обратного чтения неполны, остановитесь перед обновлением. BPM не предоставляет нумерацию курсора, изоляцию снимков, блокировки сравнения, веб-перехватчики или контракт безопасного воспроизведения для рабочих процессов чтения.

Продукт внешнего управления имеет воспроизводимый снимок извлечения/списка/чтения и знает, какие свидетельства принадлежат BPM, а какие свидетельства принадлежат внешней системе.

Продолжайте выполнять проверку перед применением или сравнение/обновление только после того, как внешняя система запишет выбранный профиль идентификаторы и версии.

Проверка и применение обновления профиля с параметром `expected_revision`

Запустите рабочий процесс обновления с первой проверкой, который защищает состояние BPM с помощью текущей версии и останавливается на устаревших или недействительных данных.

Внешний продукт уже прочитал целевой профиль и сохранил его текущий `revision`. Подготовьте документ-кандидат `Firefox policies.json` перед отправкой какого-либо исправления профиля.

В этом рабочем процессе используются API-VAL-001, API-PROFILE-003 и API-PROFILE-005. Он документирует текущий оптимистичный путь обновления; BPM не предоставляет ETags, заголовки предусловий, ключи идемпотентности или транзакции с несколькими операциями.

1. Создайте потенциальный документ Firefox вне BPM.

Запишите нужные ключи политики, исходную систему, рецензента, заявку и канал схемы.

Пример кандидата: `{"document":{"policies":`

`{"DisableTelemetry":true,"BlockAboutConfig":true}}}`.

2. Проверьте кандидата перед исправлением профиля.

Пример запроса: `POST $BPM_BASE_URL/api/validate/release-153`. Пример ответа: 200

`{"ok":true,"profile":"release-153"}`. Примеры ошибок: 400, 404, 422 или 503.

3. Перечитайте профиль непосредственно перед применением обновления, если окно утверждения длинное.

Пример запроса: `GET $BPM_BASE_URL/api/profiles/42`. Пример ответа: 200 `ProfileRead`. Если `revision` изменился, повторно запустите сравнение и утверждение, а не исправляйте устаревшее состояние.

4. Исправляет только изменяемые поля с наблюдаемой версией.

Пример запроса: `PATCH $BPM_BASE_URL/api/profiles/42` с JSON `{"flags":`

`{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":`

`{"source":"control-product","decision":"approved"}}`. Пример ответа: 200 `ProfileRead` с увеличенным `revision`.

5. Рассматривайте конфликтные реакции как сигналы остановки.

Примеры ошибок: 400 недопустимая полезная нагрузка, 404 отсутствует профиль, 409 устаревший `expected_revision` и ошибки запроса на уровне схемы 422. 409 означает, что внешнее желаемое состояние необходимо пересчитать.

6. Записывайте доказательства обновления и избегайте слепого повтора.

Внимание:

Не повторяйте тот же PATCH вслепую. Сохраняйте старую версию, новую версию, текст проверки, хеш тела исправления, статус ответа, одобрение рецензента и решение о повторной попытке вне BPM.

Обновление профиля применяется только после проверки и проверки версий с явным аудиторским свидетельством успеха или конфликта.

Экспортируйте профиль или передайте метаданные соответствия только после того, как профиль обратного считывания соответствует утвержденному желаемому состоянию.

Запуск импорта, проверки, экспорта и передачи метаданных соответствия

Подключите продукт внешней проверки или соответствия требованиям к импорту и экспорту BPM API, не рассматривая метаданные BPM как контент развертывания Firefox.

Установите `$BPM_BASE_URL` и подготовьте проверенный исходный документ `Firefox policies.json`, а также внешние метаданные соответствия, такие как сканер, билет, рецензент и решение.

В этом рабочем процессе используются API-FF-001, API-PROFILE-003, API-VAL-001 и API-FF-002. BPM хранит непрозрачные метаданные соответствия в профиле, но экспортированный Firefox JSON содержит только развертываемый `policies`.

1. Импортируйте проверенный исходный документ в профиль BPM.

Пример запроса: `POST $BPM_BASE_URL/api/profiles/import/firefox/policies.json` с JSON `{"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true}}}`. Пример ответа: 201 ProfileRead.

2. Перечитайте созданный профиль и отделите поля BPM от внешних данных.

Пример запроса: `GET $BPM_BASE_URL/api/profiles/42`. Пример ответа: 200 ProfileRead с `revision`, `flags` и непрозрачным `compliance`. Примеры ошибок: 404 или 422.

3. Перед экспортом просмотрите набор нормализованной политики.

Рецензент проверяет импортированные ключи, канал схемы, хэш исходного документа, метаданные соответствия и любые предупреждения нормализации BPM. BPM не сертифицирует результаты внешнего сканирования или принудительное исполнение Firefox.

4. Проверьте документ развертывания, который будет экспортирован.

Пример запроса: `POST $BPM_BASE_URL/api/validate/release-153` с JSON `{"document":{"policies":{"DisableTelemetry":true}}}`. Пример ответа: 200 `{"ok":true}`. Примеры ошибок: 400, 404, 422 или 503.

5. Экспортируйте документ канонических политик Firefox.

Пример запроса: `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`. Пример ответа: 200 `application/json` с `Content-Disposition` и `policies` верхнего уровня. Метаданные соответствия не экспортируются.

6. Запишите запись передачи обслуживания перед последующим развертыванием.

Внимание:

Сохраните хеш источника, профиль ID, версию, ответ проверки, хэш экспорта, результат сканирования, рецензента и нижестоящего владельца за пределами BPM. Не рассматривайте экспорт BPM как доказательство того, что Firefox применил политику.

Внешний продукт получает канонический документ о развертывании Firefox и отдельную запись о передаче соответствия с четкими границами владения.

Используйте инструменты развертывания вне BPM, чтобы разместить экспортированный `policies.json` в управляемых системах Firefox.

Запуск продукта управления воротами с работоспособностью и готовностью BPM

Используйте текущие проверки работоспособности в качестве шлюза запуска, прежде чем внешний продукт управления начнет синхронизацию API.

Развертывание исходного кода BPM уже начато. Настройте внешний продукт с помощью `$BPM_BASE_URL`, времени ожидания, интервала опроса, максимального ожидания и действия, которое необходимо предпринять в случае сбоя шлюза.

В этом рабочем процессе используются API-HEALTH-001 и API-HEALTH-002. Зонды представляют собой только небольшие предполетные сигналы; они не предоставляют аутентификацию, авторизацию, диагностику базы данных, диагностику кэша схемы, запрос идентификаторы или цели уровня обслуживания. В этом наборе модулей руководство описаны запуск с контролем работоспособности, владение данными, восстановление после сбоя и правило не повторять слепые попытки, если данные устарели.

Доступность помощника по документации отделена от работоспособности и готовности ядра BPM. В версии 0.9.3 на любой вопрос помощнику выводится опубликованное уведомление об обучении локальной модели; модель, поиск, индекс RAG и запросы к внешним источникам не запускаются. Эта граница не блокирует детерминированный поиск по документации и не должна быть шлюзом для изменения профилей или синхронизации API.

1. Проверьте базовую работоспособность перед открытием очереди синхронизации.

Пример запроса: `GET $BPM_BASE_URL/health`. Пример ответа: `200 {"status": "ok"}`. Ошибки транспорта обрабатываются внешним продуктом, поскольку для этого зонда не объявлено тело ошибки приложения BPM.

2. Проверьте готовность перед вызовом профиля или проверки.

Пример запроса: `GET $BPM_BASE_URL/health/ready`. Пример ответа: `200 {"status": "ready", "ready": true}`. Если `ready` не `true`, удерживайте очередь и не запускайте мутации.

3. Запишите решение о воротах запуска.

Сохраните временную метку, метку базового URL-адреса, время ожидания, статус проверки, наблюдаемое тело, версию контрольного продукта и выбранное следующее действие. Не храните частные узлы в примерах общих модулей руководство.

4. Начните с инвентаризации, доступной только для чтения, после прохождения ворот.

Вызовите `GET $BPM_BASE_URL/api/profiles?limit=50` перед операциями импорта, экспорта, проверки или исправления. Благодаря этому первоначальная синхронизация остается наблюдаемой и восстанавливаемой.

5. Переходите к рабочим процессам мутации только после успешного выполнения вызовов только для чтения.

Продолжайте проверять рабочие процессы перед применением, импортом-проверкой-экспортом или экспортом в соответствии с внешним графиком продукта. Сохраняйте политику повторных попыток и отсрочки за пределами BPM.

6. Закрытие при сбое, если шлюз неоднозначен.

Внимание:

Успешная проверка не резервирует изменения и не гарантирует последующий успех API. Неудачное или неоднозначное зондирование должно остановить автоматические мутации до тех пор, пока оператор не проверит журналы, состояние развертывания источника и границы сети/CORS.

Продукт внешнего управления начинает синхронизацию только тогда, когда BPM отвечает на текущее подтверждение работоспособности и готовности.

Если шлюз запуска не работает, воспользуйтесь разделами Администратор/DevOps по устранению неполадок и развертыванию исходного кода, а не добавляйте недокументированные предположения проверки.

Восстановление после сбоев интеграции и запись доказательств аудита

Обработывайте ошибки проверки, импорта/экспорта, готовности и изменений без небезопасных повторных попыток или скрытых предположений о транзакциях.

Используйте этот рабочий процесс, когда внешний продукт получает неверный ответ BPM API, тайм-аут, неожиданное тело ответа, устаревший `expected_revision` или несовпадающее свидетельство экспорта.

Этот рабочий процесс соединяет API-HEALTH-001, API-HEALTH-002, API-PROFILE-003, API-PROFILE-005, API-FF-001, API-FF-002 и API-VAL-001. Он определяет показания оператора; BPM не обеспечивает распределенную трассировку, автоматический откат, массовое восстановление транзакций или уведомления веб-перехватчиков.

1. Классифицируйте ошибку перед повторной попыткой.

Запишите, является ли сбой работоспособностью/готовностью, проверкой, импортом, экспортом, чтением, исправлением, конфликтом, тайм-аутом или транспортировкой. Сохраните статус HTTP, например 400, 404, 409, 415, 422 или 503.

2. Заморозить внешнее желаемое состояние и наблюдение BPM.

Сохраните метод запроса, путь, очищенный хеш полезной нагрузки, профиль ID, наблюдаемый `revision`, канал схемы, текст ответа, время ответа и запуск управляющего продукта ID. Никогда не включайте секреты или частные имена хостов в общие доказательства.

3. Повторно проверьте работоспособность BPM и перечитайте профиль, когда сбой можно будет устранить.

Примеры запросов: GET `$BPM_BASE_URL/health`, GET `$BPM_BASE_URL/health/ready` и GET `$BPM_BASE_URL/api/profiles/42`. Если редакция профиля изменилась, вернитесь к сравнению и утверждению.

4. Подтверждайте или экспортируйте снова только после решения, одобренного оператором.

Для проверки повторите POST `$BPM_BASE_URL/api/validate/release-153` с тем же документом-кандидатом. Для экспорта повторите GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` и сравните хэш экспортированного документа.

5. Остановить повторные попытки мутации при устаревшей версии или неоднозначных данных.

Если `expected_revision` возвращает 409, если хэш экспорта отличается или если изменились ошибки проверки, не продолжайте старый запуск. Откройте новый внешний прогон с новым прочтением, сравнением и одобрением рецензента.

6. Опубликуйте запись аудита во внешней системе записей.

Внимание:

ВРМ хранит состояние профиля, но не сохраняет полный контрольный журнал интеграции. Внешний продукт должен записывать владельца, решение, заявку, количество повторов, причину остановки, старые и новые версии, хэши и примечания оператора.

Сбои можно устранить только посредством явного подтверждения, новых операций чтения и утвержденных оператором решений о повторных попытках, а не путем слепого воспроизведения.

После восстановления свяжите запись аудита с успешной редакцией профиля или отметьте запуск интеграции как остановленный.

Устранение неполадок и оценка готовности к эксплуатации

Устранение неполадок при сбое запуска и недоступных зондах

Диагностика исходного процесса BPM, который не запускается или не отвечает на запросы работоспособности и готовности.

Используйте эту процедуру, если происходит сбой `make dev, uvicorn app.main:app --reload --port 8000, GET /health` или `GET /health/ready` во время развертывания исходного кода или запуска интеграции.

Эта диагностика разделяет процесс, среду и данные зонда. Он не создает модуль `systemd`, службу Windows, обратный прокси-сервер, конечную точку TLS или политику перезапуска производства.

1. Запишите ошибочную команду и основной журнал.

Запустите или скопируйте точную команду, затем запишите `stdout` и `stderr` из терминала переднего плана. Включите `make dev, uvicorn app.main:app --reload --port 8000`, сводку трассировки Python, рабочий каталог и версию исходного кода из `git rev-parse --short HEAD`.

2. Перед изменением кода проверьте настроенный хост и порт.

Проверьте `BPM_HOST`, `BPM_PORT`, `BPM_RELOAD`, `BPM_DATABASE_URL` и `BPM_DOCUMENTATION_SITE_DIR`. Целенаправленный повтор: `curl -fsS http://127.0.0.1:8000/health` и `curl -fsS http://127.0.0.1:8000/health/ready`.

3. Отделить запуск процесса от доступности зонда.

Если процесс не запущен, сначала исправьте зависимости, среду или свидетельства миграции. Если процесс запущен, но зонды недоступны, проверьте локальный брандмауэр, адрес привязки, переадресацию портов WSL и то, нацелен ли запрос на `127.0.0.1`, `0.0.0.0` или маршрут узла Windows.

4. Сохраняйте данные пользователя при сборе доказательств.

Копируйте только журналы, имена сред и ответы зондов. Не редактируйте сгенерированные артефакты, строки базы данных, файлы кэша схемы или исходный код в рамках обычного восстановления. Не удаляйте `data/bpm.db`, чтобы пройти запуск.

5. Выберите наименьший путь повторного запуска и эскалации.

Целенаправленный повтор: для дыма продукта, только в том случае, если задействован вывод документации, и только для контрактов документации.

Внимание:

Если зонды остаются недоступными после проверок конфигурации и процессов, остановите автоматические запуски интеграции и перейдите к эскалации с использованием команд, журналов, имен конфигураций и доказательств зондирования.

Оператор может отличить сбойный процесс, недостижимую пробу и границу развертывания без изменения пользовательских данных.

После успешного завершения проверок вернитесь к теме запуска с контролем работоспособности или примеров многократного использования API, прежде чем запускать изменяющиеся интеграции.

Устранение ошибок кэша схемы и ошибок проверки API

Диагностика недоступного кэша схемы, неизвестного канала схемы и ошибок проверки политик Firefox без исправления созданных артефактов схемы.

Используйте эту процедуру, когда `POST $BPM_BASE_URL/api/validate/release-153`, `POST $BPM_BASE_URL/api/validate/beta-999` или пользовательская проверка возвращает 400, 404, 422 или 503; необъектный документ может вернуть `200 ok=false` с помощью `Expected object with policy mappings`.

Проверка BPM зависит от поддерживаемых каналов схемы и локальной конфигурации схемы/кэша. Диагностика записывает, связана ли проблема с формой запроса, неизвестным профилем, неверными данными политики Firefox или недоступным кэшем схемы.

1. Запишите точный запрос проверки и ответ.

Сохраните очищенный хеш тела запроса, канал схемы, статус HTTP, `detail`, `message`, `error` и `issues`. Храните кандидата `policies.json` вне журналов, если он содержит значения клиентов.

2. Сначала классифицируйте ошибки формы запроса.

Необъектный `document` или отсутствующий `document.policies` — это ввод оператора, а не дефект кэша схемы; текущий BPM может сообщить об этом как `200 ok=false`.

Целенаправленный повтор: `POST $BPM_BASE_URL/api/validate/release-153` с `{"document":{"policies":{"DisableTelemetry":true}}}`.

3. Классифицируйте сбои канала схемы и кэша.

Профиль, такой как `beta-999`, может возвращать 404; недоступная зарегистрированная схема может вернуть 503. Проверьте `BPM_SCHEMA_BASE_URL`, `BPM_SCHEMA_CACHE_DIR` и `BPM_SCHEMA_HTTP_TIMEOUT` перед более широкой перестройкой.

4. Используйте поддерживаемые проверки обновления и проверки.

Целенаправленный повтор: для изменений документации API или тестов схемы продукта при изменении поведения API. Не редактируйте вручную `app/schemas/cache` или созданные скелеты Firefox.

5. Прекратите небезопасное восстановление и сохраните доказательства.

Если один и тот же допустимый запрос меняет статус при разных запусках, сохраните тела ответов, исходную версию, путь к кэшу и переменные среды.

Внимание:

Не редактируйте сгенерированные артефакты схемы или строки базы данных в ходе обычного восстановления; отделять дефекты продукта от ошибок полезной нагрузки оператора.

У оператора произошла классифицированная ошибка проверки с наименьшим повторным запуском и без скрытой мутации схемы.

После успешного завершения проверки вернитесь к рабочим процессам «проверка перед применением» или «импорт-проверка-экспорт».

Устранение неполадок импорта и экспорта Firefox

Диагностика ошибок импорта JSON, составного импорта и экспорта политик Firefox, сохраняя при этом исходные и экспортированные доказательства.

Используйте эту процедуру, когда POST

`$BPM_BASE_URL/api/profiles/import/firefox/policies.json` или GET

`$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` возвращает 400, 404, 409, 415 или 422.

Устранение неполадок импорта/экспорта отделяет достоверность исходного документа, форму составного поля, повторяющиеся имена профилей, доступ к архивному профилю и канонический экспорт Firefox от последующих инструментов развертывания.

1. Определите операцию и тип контента.

Запишите, был ли запрос `JSON FirefoxPoliciesJsonImportRequest`, `multipart/form-data` с `file=@policies.json` или экспортирован с помощью `download=1`, `pretty=1` или `include_deleted=true`.

2. Проверьте исходный документ Firefox перед повторной попыткой импорта.

Целенаправленный повтор: POST `$BPM_BASE_URL/api/validate/release-153` с тем же `document.policies`. Если проверка не удалась, исправьте исходный документ вне BPM перед следующим импортом.

3. Проверьте ошибки оператора импорта.

Для 409 выберите новое синтетическое имя профиля или намеренно заархивируйте/восстановите его. Для 415 проверьте `application/json` или составные поля загрузки. Для 422 запишите ошибки на уровне поля и не перезаписывайте хранилище профиля напрямую.

4. Проверьте состояние экспорта.

Для 404 перечитайте GET `$BPM_BASE_URL/api/profiles/42?include_deleted=true`. Для заархивированных профилей решите, подходит ли `include_deleted=true`. Сравните хеш экспортированного документа с хешем проверенного источника; Метаданные профиля BPM и метаданные соответствия не экспортируются.

5. Запишите условия восстановления и остановки.

Целенаправленный повтор: для документированных примеров.

Внимание:

Не повторяйте попытку вслепую после несоответствия данных импорта/экспорта; сохранить хеш источника, хэш экспорта, профиль ID, версию, статус HTTP и решение оператора.

Оператор может определить, относится ли ошибка к исходному JSON, составным полям, состоянию жизненного цикла профиля или свидетельству экспорта.

После восстановления прикрепите доказательства к записи внешнего аудита контрольного продукта.

Устранение проблем с базой данных и хранилищем

Диагностируйте локальные проблемы SQLite, пути хранения, резервного копирования и видимости профиля без непосредственного редактирования строк базы данных.

Используйте эту процедуру, когда профили исчезают, происходит сбой записи, данные миграции неясны, контрольные суммы резервных копий не совпадают или BPM_DATABASE_URL указывает на неожиданное расположение.

Текущее развертывание исходного кода использует локальное хранилище, такое как `sqlite+aiosqlite:///./data/bpm.db`, и файлы, принадлежащие репозиторию в `data/`, пути к кэшу схемы и пути к сайту документации. В BPM нет управляемого резервного копирования или оркестрации восстановления.

1. Подтвердите URL-адрес активной базы данных и рабочий каталог.

Запишите BPM_DATABASE_URL, текущий каталог, исходную версию и укажите, запускается ли процесс из файловой системы Linux или `/mnt/c`. Не предполагайте, что две оболочки указывают на один и тот же `data/bpm.db`.

2. Перед открытием файлов хранилища проверьте данные миграции и API профиля.

Целенаправленный повтор: `alembic upgrade head`, затем `curl -fsS http://127.0.0.1:8000/api/profiles?limit=50`. Если API возвращает ожидаемые профили, признаком может быть состояние фильтра, жизненного цикла или браузера.

3. Перед разрушительным восстановлением проверьте доказательства резервной копии.

Проверьте записи `mkdir -p backups`, `cp data/bpm.db backups/bpm-pre-update.db` и `sha256sum backups/bpm-pre-update.db`. Сохраняйте экспорт критически важных профилей `policies.json` в качестве отдельного доказательства.

4. Избегайте прямого редактирования строк базы данных.

Не редактируйте строки SQLite, сгенерированные артефакты или файлы кэша схемы в ходе обычного восстановления. Вместо этого используйте документированный архив профиля, процедуры восстановления, импорта, экспорта или обновления из исходного кода.

5. Если восстановление небезопасно, сообщите об этом с доказательствами хранения.

Укажите URL-адрес базы данных, путь к файлу, контрольную сумму, команду миграции, ответ API и свидетельства резервного копирования/экспорта.

Внимание:

Если проверенная резервная копия до сбоя не существует, остановитесь перед удалением или заменой `data/bpm.db`; BPM не обеспечивает восстановление на определенный момент времени.

У оператора есть диагностика хранилища, которая сохраняет пользовательские данные и отделяет состояние базы данных от UI или симптомов фильтра.

После того, как хранилище станет стабильным, повторно запустите работоспособность/готовность и рабочий процесс API с наименьшей затронутой проблемой.

Устранение неполадок сети WSL и устаревших зависимостей

Диагностика Windows 10/11 с помощью проблем доступа к WSL, размещения файловой системы Linux и устаревших зависимостей Python после изменения исходного кода.

Используйте эту процедуру, если Windows не может подключиться к процессу BPM, размещенному в WSL, зонды работают внутри WSL, но не из Windows, производительность файлов нестабильна в /mnt/с или зависимости кажутся устаревшими после `git checkout <approved-new-ref>`.

При устранении неполадок WSL разделяется доступ к браузеру Windows, привязка процессов Linux, размещение файловой системы и обновление среды Python. BPM не предоставляет встроенную службу Windows или установщик в версии 0.9.1.

1. Подтвердите, где находятся исходный код и виртуальная среда.

Для оформления заказа выберите файловую систему Linux и `.venv`. Запишите, находятся ли файлы в папке /mnt/с, что может изменить производительность и разрешения. Не смешивайте несколько проверок во время диагностики.

2. Проверьте привязку процесса WSL и маршрут Windows.

Проверьте `BPM_HOST="0.0.0.0"`, `BPM_PORT="8000"`, `curl -fsS http://127.0.0.1:8000/health` внутри WSL и доступ к браузеру из Windows. В случае сбоя Windows запишите доказательства брандмауэра и переадресации портов.

3. Обновить зависимости после перемещения исходного кода или изменения версии.

Целенаправленный повтор: `source .venv/bin/activate`, `python -m pip install --upgrade pip` и `pip install -e ".[dev]"`. Если ошибки зависимостей сохраняются, запишите имена пакетов и версию исходного кода, а не редактируйте установленные пакеты вручную.

4. Отделяйте симптомы устаревших зависимостей от проблем схемы или базы данных.

Если импорт/экспорт или проверка завершаются неудачно после обновления зависимостей, используйте схему/кэш и средство устранения неполадок импорта/экспорта. Если видимость профиля изменится, воспользуйтесь средством устранения неполадок базы данных/хранилища.

5. Откажитесь от небезопасных собственных предположений Windows.

Не создавайте недокументированные службы Windows, не перемещайте базу данных между файловыми системами и не заявляйте о собственном поведении установщика.

Внимание:

Если сеть WSL остается неоднозначной, остановите внешнюю интеграцию и перейдите к эскалации с использованием дистрибутива WSL, пути оформления заказа, адреса привязки, URL-адреса Windows и проверки выходных данных.

Оператор может распознавать дефекты маршрутизации WSL, размещения файловой системы и обновления зависимостей без изменения хранилища продукта.

После того, как доступ WSL стабилизируется, вернитесь к проверке развертывания источника и запуску с контролем работоспособности.

Устранение недоступности портала документации

Соберите состояние `/help/` и сведения о развёртывании, если установленный портал документации недоступен.

Используйте эту процедуру, если установленный экземпляр BPM сообщает для `/help/` об отсутствующем, устаревшем, несовместимом, неполном или не найденном состоянии.

Портал документации является частью интерфейса BPM. Его доступность зависит от соответствия установленного артефакта документации запущенной версии BPM; BPM не восстанавливает отсутствующий или несовместимый артефакт во время работы.

1. Запишите URL портала и отображаемое состояние.

Сохраните полный URL `/help/`, локаль, сообщение браузера, версию BPM и время сбоя. Не смешивайте это состояние с FastAPI `/docs` и `/openapi.json`: это отдельные интерфейсы.

2. Подтвердите запущенный экземпляр BPM и его настройку документации.

Запишите хост и порт процесса, ревизию исходного кода или идентификатор развёрнутого релиза, а также значение `BPM_DOCUMENTATION_SITE_DIR`, не копируя секреты.

3. При неготовом состоянии остановите зависимые действия с документацией.

Внимание:

Не считайте отсутствующий или несовместимый портал работающей системой справки и не редактируйте установленные файлы вручную, чтобы обойти это состояние.

4. Передайте доказательства владельцу развёртывания BPM.

Передайте URL портала, отображаемое состояние, версию BPM, имя настройки сайта документации, журналы процесса и время наблюдения. Владелец развёртывания восстановит соответствующий артефакт по поддерживаемой процедуре документации.

5. Проверьте портал после восстановления.

После восстановления соответствующего артефакта снова откройте `/help/` и затронутые маршруты контекстной справки.

Оператор имеет достаточно сведений, чтобы отличить недоступный артефакт документации от API BPM и запросить корректное восстановление.

Зафиксируйте восстановленное состояние портала вместе с исходными сведениями эскалации.

Проверка сетевой доступности и готовности к работе через прокси

Подготовьте заметки о рисках в сети и обратные вопросы без публикации неподдерживаемых TLS или прокси-рецептов, как поведение BPM.

Текущая оценка готовности источника с одним узлом завершена, и оператор знает, получают ли пользователи доступ к BPM через петлю, переадресованный адрес WSL, лабораторный хост или внешний прокси-сервер.

BPM может работать на настроенном хосте и порту, но документация BPM не предоставляет официальный обратный прокси-сервер, завершение TLS, шлюз аутентификации, политику заголовка или рецепт ограничения скорости.

1. Выберите наименьшее воздействие на сеть, соответствующее текущей оценке.

Предпочитаю 127.0.0.1. Используйте `BPM_HOST="0.0.0.0"` только тогда, когда этого требует переадресация WSL или контролируемая лабораторная сеть, и запишите, почему `BPM_PORT="8000"` доступен.

2. Запишите вопросы о готовности прокси-сервера перед добавлением внешнего прокси-сервера.

Ответьте, кому принадлежат сертификаты TLS, маршрутизация имени хоста, ограничения размера запроса, значения времени ожидания, происхождение CORS через `BPM_CORS_ALLOW_ORIGINS`, пересылаемые заголовки, журналы доступа и реагирование на инциденты. Эти вопросы можно подготовить уже сейчас; официальные рецепты TLS/прокси запланированы, но еще не реализованы.

3. Убедитесь, что датчики по-прежнему проходят через выбранную границу.

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

Если в лаборатории используется внешний прокси-сервер, запишите как зонды с прямым источником, так и зонды на стороне прокси. Не рассматривайте ответ прокси-сервера 200 как доказательство аутентификации, авторизации BPM или HA.

4. Явно отмечать неподдерживаемые сетевые утверждения.

Внимание:

BPM не предоставляет пакетные службы, официальные рецепты TLS/прокси, поддерживаемую кластеризацию HA, правила WAF, подпись запросов, ограничение скорости, SSO, OAuth/OIDC, управление доступом на основе ролей или усиление безопасности производства. Оставьте их в качестве внешнего контроля или отложенных требований.

Сетевое примечание отделяет то, что можно настроить сейчас, от решений обратного прокси и TLS, которые остаются внешними или отложенными.

Используйте раздел об окнах мониторинга, резервного копирования и обновления перед запуском общего окна оценки.

Планирование входных данных для мониторинга, резервного копирования и окон обновления

Подготовьте доказательства текущего состояния экземпляра, работающего на исходном коде, избегая при этом претензий по поводу управляемого мониторинга, официальной автоматизации восстановления или последовательных обновлений.

Экземпляр прошел проверку готовности источника и доступности сети, и за окно оценки отвечает один администратор или владелец DevOps.

До тех пор, пока не будут приняты решения по распределению и обслуживанию во время выполнения, планирование операций должно быть сосредоточено на наблюдаемых входных данных, ручном резервном копировании и экспорте данных, а также на явных окнах обновления для одного исходного узла.

1. Перечислите существующие на сегодняшний день входные данные мониторинга.

Запись состояния процесса, выходных данных терминала переднего плана, `stdout`, `stderr`, `GET /health`, `GET /health/ready`, результатов выбранных примеров API, доступа к документации через `/help/` и исходной версии из `git rev-parse --short HEAD`.

2. Соберите резервную копию и экспортируйте доказательства перед окном.

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

Также экспортируйте репрезентативные документы Firefox, такие как `GET $BPM_BASE_URL/api/export/profiles/42/firefox/policies.json`, и сохраните полученные доказательства `policies.json` с контрольными суммами.

3. Запланируйте окно обновления «останови мир» для изменений исходного кода.

Для версии 0.9.1 запланируйте время простоя одного узла до `git fetch --tags`, обновления зависимостей, `alembic upgrade head`, и . Последовательные обновления запланированы, но еще не реализованы.

4. Отмечайте пробелы как препятствия, а не скрытые предположения.

Внимание:

BPM не обеспечивает управляемый мониторинг, автоматическое планирование резервного копирования, официальную автоматизацию восстановления, восстановление на определенный момент времени, управляемые секреты, ротацию токенов или оркестрацию чередующихся обновлений. Используйте подтверждение вручную и условия остановки из модуля руководство обновления.

План операции содержит входные данные для мониторинга, данные резервного копирования/экспорта и реалистичное окно обновления для текущей операции с источником.

Прикрепите это свидетельство к НА и записи отложенной границы, чтобы в дальнейшей работе по распространению можно было повторно использовать тот же контрольный список.

Помощник BPM по документации

Эксплуатация помощника по документации в BPM 0.9.3

Проверьте границу функции с уведомлением об обучении, не делая BPM и детерминированный поиск зависимыми от чата.

Используйте поддерживаемую копию исходного кода BPM 0.9.3 и базовые ресурсы из [Минимальные системные требования для BPM](#).

В этой версии чат является только локализованным интерфейсом для читателя. Он не запускает и не устанавливает модель, не извлекает документацию, не использует индекс E5, не генерирует ответ, не показывает цитаты и не обращается к внешним источникам. Детерминированный поиск по документации остаётся отдельным и доступным.

1. Запустите BPM обычным поддерживаемым способом из исходного кода.

Не добавляйте к запуску BPM настройку модели, индекса, эмбедингов или внешнего провайдера для версии 0.9.3.

2. Откройте портал документации на языке читателя.

Убедитесь, что помощник открывается из правого нижнего угла и не изменяет элементы обычного поиска.

3. Отправьте вопрос по BPM.

В переписке должно появиться локализованное уведомление о том, что обучение локальной модели выполняется и функция будет доступна в одной из следующих версий.

4. Отправьте вопрос не по теме BPM.

Ожидается то же уведомление: в этой версии чат не является помощником общего назначения.

5. Найдите актуальный ответ детерминированным поиском по документации.

Поиск, фильтры, навигация и порядок результатов остаются поддерживаемым способом получения сведений о продукте.

Помощник показывает выпущенное локализованное уведомление об обучении, а BPM и детерминированный поиск остаются независимыми и работоспособными.

Не устанавливайте модель и не создавайте RAG-генерацию для 0.9.3. Если уведомление, локаль или граница с обычным поиском отличаются от описанного поведения, зарегистрируйте дефект документации.

Поддержка границы помощника по документации в BPM 0.9.3

Поддерживайте корректность локализованного уведомления об обучении и границы с детерминированным поиском при обновлении документации BPM.

Подготовьте согласованную ревизию исходного кода BPM и рабочий процесс публикации документации.

В версии 0.9.3 нет локальной модели, артефакта модели, RAG-генерации, хранилища эмбеддингов, хранилища цитат или провайдера внешних источников, которые нужно поддерживать. Не создавайте состояние обслуживания для функций, не поставляемых в этом релизе.

1. Примените согласованную ревизию BPM и пересоберите документацию.

Используйте обычный процесс документации, включая `make docs-install-dev` при подготовке среды разработки.

2. Откройте опубликованную документацию на всех поддерживаемых языках.

Убедитесь, что точка входа помощника доступна, а детерминированный поиск работает независимо.

3. Отправьте по одному вопросу на каждом языке.

Проверьте уведомление для соответствующей локали и отсутствие ответа, источника, цитаты, внешнего утверждения и установки модели.

4. Не запускайте для 0.9.3 команды установки модели или создания RAG-индекса.

Они относятся к следующему релизу и не являются способом восстановления уведомления об обучении.

5. Зафиксируйте ревизию документации и результат проверки.

Несоответствие между выпущенным уведомлением и границей детерминированного поиска оформляйте как дефект документации.

Опубликованный портал сохраняет локализованное уведомление в чате и отдельный путь к актуальным сведениям через детерминированный поиск.

Не храните вопросы пользователей как артефакт обслуживания. Пересматривайте операции с моделью и поиском только после их явной поставки в следующем релизе.